Akademia Górniczo-Hutnicza
im. Stanisława Staszica w Krakowie

Wydział Elektrotechniki, Automatyki, Informatyki
i Inżynierii Biomedycznej (EAIiIB)

ForVis Desktop

Pełna Dokumentacja Techniczna Systemu

Wersja 1.0

Autorzy: Kevin Stuka
Kalina Rączka

Opiekun projektu: dr hab. inż. Radosław Klimek

Kraków, Polska 2026

Spis treści

1 Informacje wstępne **4**
1.1 Identyfikacja systemu 4
1.2 Cel i zakres dokumentu 4
1.3 Definicje, akronimy i skróty 4

2 Opis ogólny i kontekst teoretyczny **6**
2.1 Cel biznesowy i naukowy systemu 6
2.2 Problem spełnialności i format danych 6
2.2.1 Koniunkcyjna postać normalna (CNF) 6
2.2.2 Format plików DIMACS CNF 7
2.3 Użytkownicy systemu i otoczenie 8

3 Architektura systemu **9**
3.1 Charakterystyka ogólna 9
3.2 Diagram architektury 9
3.3 Stos technologiczny i komponenty fizyczne 10
3.3.1 Warstwa prezentacji (Electron + Angular) 10
3.3.2 Warstwa logiki biznesowej (Django Embedded) 10
3.3.3 Warstwa danych (SQLite) 11
3.3.4 Warstwa przetwarzania asynchronicznego (Redis + Celery) 11
3.4 Komunikacja międzyprocesowa i zarządzanie 11
3.4.1 Skrypt nadzorczy (Supervisor) 11
3.4.2 Kanały komunikacji 11
3.5 Topologia wdrożenia 12

4 Interfejsy i komunikacja **13**
4.1 Wewnętrzny interfejs REST API 13
4.1.1 Adres bazowy i konfiguracja 13
4.2 Specyfikacja punktów końcowych (Endpoints) 13
4.2.1 Autoryzacja i zarządzanie sesją 13
4.2.2 Zarządzanie plikami wejściowymi (SAT/CNF) 14
4.2.3 Zarządzanie plikami Max-SAT 14
4.2.4 Sterowanie procesem wizualizacji 14
4.2.5 Dostęp do danych wynikowych 15
4.3 Komunikacja międzyprocesowa (IPC) 15
4.3.1 Uruchamianie i nadzór (Spawn) 15
4.3.2 Przekierowanie strumieni (Logging) 15
4.3.3 Sygnały sterujące (Graceful Shutdown) 15
4.4 Formaty wymiany danych 16

5 Opis modułów wizualizacyjnych **17**
5.1 Interaction Graph . . . 17
5.2 Factor Graph . . . 18
5.3 Resolution graph . . . 19
5.4 Tree visualization . . . 20
5.5 Cluster Visuzalization . . . 21
5.6 Direct Graphical Model . . . 22
5.7 2-Clauses Interaction Graph . . . 24
5.8 DPLL Solver Visualization . . . 24
5.9 Heatmap Visualization . . . 25
5.10 Matrix Visualization . . . 26

6 Mechanizmy przetwarzania i sterowania obliczeniami **28**
6.1 Środowisko wykonawcze Celery na Windows . . . 28
6.1.1 Model wykonawczy `solo` . . . 28
6.2 Mechanizm Stop/Resume (Application-Level Preemption) . . . 29
6.2.1 Strategia Time-Based Throttling . . . 29
6.3 Kategoryzacja zadań wizualizacyjnych . . . 29
6.3.1 Kategoria 1: Zadania standardowe (Przerywalne) . . . 29
6.3.2 Kategoria 2: Zadania natychmiastowe (Instant) . . . 30
6.3.3 Kategoria 3: Wyjątek algorytmiczny (Solver DPLL) . . . 30
6.4 Obsługa błędów obliczeniowych . . . 30
6.5 Przepływ . . . 30

7 Bezpieczeństwo, niezawodność i diagnostyka systemu **32**
7.1 Mechanizmy zapewnienia integralności i bezpieczeństwa danych . . . 32
7.1.1 Sanityzacja danych wejściowych (Safe Evaluation) . . . 32
7.1.2 Walidacja formatów DIMACS/CNF . . . 33
7.2 System obsługi błędów krytycznych (Fault Tolerance) . . . 33
7.2.1 Kapsułkowanie procesu wykonawczego . . . 33
7.3 Logowanie strumieniowe (Stdout/Stderr) . . . 33

8 Instalacja, konfiguracja i proces wdrożenia **34**
8.1 Wdrożenie produkcyjne (Użytkownik końcowy) . . . 34
8.1.1 Wymagania dla użytkownika końcowego . . . 34
8.1.2 Procedura uruchomienia . . . 34
8.2 Konfiguracja środowiska deweloperskiego (Dla programistów) . . . 35
8.2.1 Wymagania programistyczne . . . 35
8.2.2 Instalacja zależności . . . 35
8.3 Proces budowania wersji dystrybucyjnej . . . 36

9 Testowanie i walidacja systemu **37**
9.1 Testy jednostkowe i integracyjne (Backend) . . . 37
9.1.1 Weryfikacja parserów . . . 37
9.1.2 Weryfikacja struktur grafowych . . . 37
9.2 Testy funkcjonalne i akceptacyjne (UI/UX) . . . 38
9.3 Testy wydajnościowe i weryfikacja wywłaszczania . . . 38
9.3.1 Klasyfikacja wizualizacji pod kątem wywłaszczania . . . 38
9.3.2 Podsumowanie . . . 39

10 Implementacja nowych formuł wizualizacji **40**
10.1 CDCL Solver – wizualizacja działania solvera SAT z uczeniem na konfliktach 40
10.1.1 Cel i zakres metody 40
10.1.2 Implementacja Algorytmu CDCL 40
10.1.3 Heurystyki Wyboru Zmiennych 40
10.1.4 Wizualizacja 41
10.2 Kontrfaktyczna wizualizacja decyzji (What-if) 42
10.2.1 Cel i zakres metody 42
10.2.2 Architektura rozwiązania 42
10.2.3 Algorytm działania 42
10.2.4 Kluczowe funkcjonalności interfejsu 43
10.3 Trudność Semantyczna Zmiennych 43
10.3.1 Cel i koncepcja 43
10.3.2 Instrumentacja Solvera (Backend) 43
10.3.3 Przetwarzanie i Agregacja danych 44
10.3.4 Wizualizacja (Frontend) 44
10.4 Wizualizacja Przestrzeni Stanów i Krajobrazu Kosztów (SAT Landscape) . 45
10.4.1 Cel Metody 45
10.4.2 Architektura Rozwiązania 45
10.4.3 Integracja z Solverem (Trajektoria) 46
10.4.4 Implementacja Techniczna 46

11 Podsumowanie i wnioski końcowe **48**

Rozdział 1

Informacje wstępne

1.1 Identyfikacja systemu

Przedmiotem niniejszej dokumentacji jest system **ForVis Desktop** w wersji **1.0**. Jest to samodzielna aplikacja desktopowa przeznaczona dla systemów operacyjnych z rodziny Windows (10/11), służąca do zaawansowanej wizualizacji i analizy formuł logicznych (SAT oraz MaxSAT).

System jest ewolucją wcześniejszego rozwiązania webowego, przekształconą w zintegrowane narzędzie typu *offline*, niewymagające od użytkownika końcowego instalacji zewnętrznych zależności ani konfiguracji środowisk serwerowych.

1.2 Cel i zakres dokumentu

Niniejszy dokument stanowi kompletną dokumentację techniczną systemu, zgodną z wytycznymi inżynierii oprogramowania. Jego celem jest:

- Przedstawienie architektury systemu w wersji Desktop, ze szczególnym uwzględnieniem integracji komponentów w środowisku Electron.
- Opisanie zaimplementowanych algorytmów wizualizacji, w tym metod opartych na teorii grafów, hipergrafach oraz uczeniu maszynowym.
- Wyjaśnienie mechanizmów sterowania procesami obliczeniowymi, w tym funkcji wstrzymywania i wznawiania zadań (Stop/Resume).
- Dostarczenie instrukcji instalacji oraz obsługi dla użytkownika końcowego.

Dokumentacja jest skierowana do deweloperów planujących dalszy rozwój aplikacji, administratorów systemów oraz użytkowników technicznych zainteresowanych wewnętrznym działaniem algorytmów solverów SAT.

1.3 Definicje, akronimy i skróty

W celu zapewnienia jednoznaczności terminologicznej, w dokumentacji stosowane są następujące pojęcia:

Termin/Skrót	Definicja
SAT	Problem spełnialności formuł logicznych (*Boolean Satisfiability Problem*).
CNF	Koniunkcyjna postać normalna (*Conjunctive Normal Form*) – standardowy sposób zapisu formuł logicznych.
DIMACS	Standardowy format plików tekstowych używany do zapisu problemów SAT i grafowych.
Electron	Framework umożliwiający tworzenie aplikacji desktopowych przy użyciu technologii webowych, pełniący w systemie rolę „wrappera" dla pozostałych komponentów.
IPC	*Inter-Process Communication* – mechanizm wymiany danych między procesami (tutaj: między Electronem a Pythonem).
Celery	Rozproszony system kolejkowania zadań, wykorzystywany w ForVis do asynchronicznego generowania wizualizacji.
Redis	Magazyn struktur danych w pamięci, pełniący rolę brokera wiadomości dla Celery (zastępujący RabbitMQ w wersji Desktop).
CDCL	*Conflict-Driven Clause Learning* – algorytm wykorzystywany w nowoczesnych solverach SAT, implementowany i wizualizowany w systemie.
GAT	*Graph Attention Networks* – sieci neuronowe z mechanizmem uwagi, stosowane do analizy istotności krawędzi w grafach logicznych.

Rozdział 2

Opis ogólny i kontekst teoretyczny

2.1 Cel biznesowy i naukowy systemu

Głównym celem systemu ForVis Desktop jest dostarczenie badaczom oraz studentom inżynierii oprogramowania narzędzia umożliwiającego intuicyjną analizę struktury formuł logicznych. Wizualizacja danych jest kluczowym elementem wspomagającym zrozumienie złożonych problemów obliczeniowych, pozwalając na szybsze dostrzeganie wzorców i zależności, które są trudne do wyekstrahowania z surowego zapisu tekstowego.

Wersja desktopowa systemu realizuje postulat dostępności i prywatności. Przeniesienie obliczeń do środowiska lokalnego (offline) eliminuje konieczność przesyłania wrażliwych danych badawczych na zewnętrzne serwery oraz uniezależnia użytkownika od dostępności łącza internetowego.

2.2 Problem spełnialności i format danych

2.2.1 Koniunkcyjna postać normalna (CNF)

Koniunkcyjna postać normalna (CNF, ang. *Conjunctive Normal Form*) to standardowa forma zapisu formuł logicznych w logice zdań. Jest ona powszechnie stosowana w teorii obliczeń, a w szczególności w algorytmach rozwiązywania problemu spełnialności formuł logicznych (SAT).

Formuła logiczna w postaci CNF zdefiniowana jest jako koniunkcja (operacja logiczna *i*, oznaczana jako $\wedge$) klauzul, gdzie każda klauzula jest alternatywą (operacja logiczna *lub*, oznaczana jako $\vee$) literałów. Literałem nazywamy zmienną logiczną lub jej negację.

Formalnie formułę $\mathcal{F}$ w postaci CNF można zapisać wzorem:

$$\mathcal{F} = \bigwedge_{i=1}^{m} C_i = \bigwedge_{i=1}^{m} \left(\bigvee_{j=1}^{k_i} l_{i,j} \right)$$

gdzie:

- C_i to i-ta klauzula,
- $l_{i,j}$ to j-ty literał w i-tej klauzuli (zmienna x lub jej negacja $\neg x$),
- $\vee$ oznacza alternatywę,

- $\wedge$ oznacza koniunkcję.

Przykład: Poniższe wyrażenie jest przykładem formuły w postaci CNF składającej się z trzech klauzul:

$$(A \vee \neg B) \wedge (\neg A \vee C \vee D) \wedge (B \vee \neg C)$$

2.2.2 Format plików DIMACS CNF

Pliki DIMACS CNF to standard przemysłowy wykorzystywany do zapisu problemów SAT. Jest to format tekstowy (ASCII), używany powszechnie w benchmarkach oraz jako wejście dla solverów SAT. Jego struktura została zaprojektowana tak, aby była łatwa do parsowania maszynowego.

Struktura pliku

Plik w formacie DIMACS CNF składa się z trzech zasadniczych części:

1. **Linie komentarza:** Rozpoczynają się od znaku `c`. Są opcjonalne i służą do zamieszczania metadanych lub opisów dla człowieka.
2. **Linia problemu (nagłówek):** Rozpoczyna się od słowa kluczowego `p`, po którym następuje format (`cnf`), liczba zmiennych oraz liczba klauzul.
3. **Lista klauzul:** Sekwencja liczb całkowitych reprezentująca poszczególne klauzule.

W tym formacie zmienne logiczne mapowane są na liczby całkowite niezerowe:

- Wartość dodatnia (np. `5`) oznacza zmienną x_5.
- Wartość ujemna (np. `-5`) oznacza negację zmiennej $\neg x_5$.
- Znak `0` (zero) oznacza koniec bieżącej klauzuli.

Przykład i interpretacja

Poniżej przedstawiono przykładową zawartość pliku DIMACS oraz jej interpretację logiczną.

```
c Przykładowa formuła w CNF
p cnf 3 3
1 -2 0
-1 3 4 0
2 -3 0
```

Interpretacja poszczególnych linii:

- `c ...` – Linia komentarza, ignorowana przez parser.
- `p cnf 3 3` – Nagłówek informujący, że formuła zawiera **3 zmienne** i **3 klauzule**.
- `1 -2 0` – Pierwsza klauzula: $(x_1 \vee \neg x_2)$.

- `-1 3 4 0` – Druga klauzula: $(\neg x_1 \vee x_3 \vee x_4)$.
- `2 -3 0` – Trzecia klauzula: $(x_2 \vee \neg x_3)$.

System ForVis Desktop wykorzystuje parsery zgodne z powyższą specyfikacją, obsługując dodatkowo walidację poprawności struktury pliku przed rozpoczęciem wizualizacji.

2.3 Użytkownicy systemu i otoczenie

Odbiorcami systemu są przede wszystkim inżynierowie oprogramowania, badacze zajmujący się logiką obliczeniową oraz studenci kierunków informatycznych. Ze względu na architekturę desktopową, system działa w całkowitej izolacji od sieci zewnętrznej. Otoczenie systemu ogranicza się do systemu operacyjnego użytkownika (Windows), lokalnego systemu plików (przechowywanie instancji CNF) oraz zasobów sprzętowych komputera (CPU/RAM wykorzystywane do obliczeń asynchronicznych).

Rozdział 3

Architektura systemu

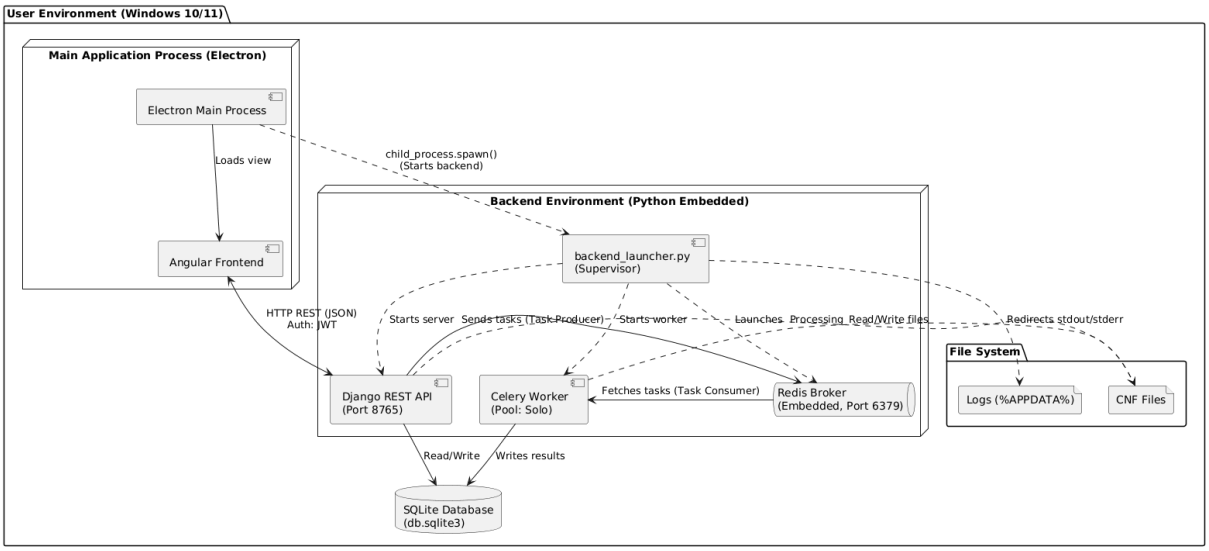
3.1 Charakterystyka ogólna

System ForVis Desktop został zaprojektowany w architekturze zintegrowanej (ang. *embedded*), w której warstwa prezentacji, logiki biznesowej oraz przetwarzania danych działają w obrębie jednej aplikacji klienckiej. Stanowi to odejście od klasycznego modelu klient-serwer na rzecz modelu desktopowego, zapewniającego pełną autonomię działania (tryb offline) oraz brak konieczności konfiguracji środowiska po stronie użytkownika końcowego.

Jako szkielet aplikacji (ang. *wrapper*) wykorzystano framework Electron, który zarządza cyklem życia procesów podrzędnych. Backend aplikacji, oparty na języku Python, jest uruchamiany jako proces tła, niewidoczny dla użytkownika, i udostępnia interfejs API na lokalnym interfejsie sieciowym (loopback).

3.2 Diagram architektury

Poniższy diagram (Rys. 3.1) przedstawia szczegółowy schemat komponentów systemu oraz kanały komunikacyjne między nimi. Wyróżniono w nim podział na środowisko uruchomieniowe Electrona (proces główny i renderujący) oraz środowisko backendowe (Python/Django), zarządzane przez dedykowany skrypt nadzorczy.



Rysunek 3.1: Schemat architektury komponentowej systemu ForVis Desktop

3.3 Stos technologiczny i komponenty fizyczne

Wybór technologii podyktowany był wymaganiami dotyczącymi przenośności na systemy Windows oraz koniecznością zachowania kompatybilności z istniejącymi algorytmami wizualizacji napisanymi w języku Python.

3.3.1 Warstwa prezentacji (Electron + Angular)

Interfejs użytkownika (UI) został zaimplementowany w frameworku **Angular**. Odpowiada on za interakcję z użytkownikiem, renderowanie wizualizacji grafowych oraz przesyłanie plików CNF. Całość osadzona jest w środowisku **Electron**, który pełni dwie funkcje:

- **Kontener aplikacji:** Udostępnia silnik renderujący Chromium, umożliwiając wyświetlanie aplikacji webowej jako natywnego okna systemu Windows.
- **Zarządca procesów:** Odpowiada za uruchomienie (spawn) oraz bezpieczne zamknięcie procesów backendowych (Python, Redis) wraz z zamknięciem okna aplikacji.

3.3.2 Warstwa logiki biznesowej (Django Embedded)

Rdzeń systemu stanowi framework **Django**, działający w trybie osadzonym. W przeciwieństwie do typowych wdrożeń webowych, tutaj serwer nie jest wystawiony do sieci publicznej, lecz nasłuchuje wyłącznie na adresie `127.0.0.1:8765`. Odpowiada on za:

- Parsowanie i walidację plików DIMACS CNF.
- Obsługę żądań HTTP REST pochodzących z warstwy prezentacji.
- Zarządzanie zlecaniem zadań obliczeniowych do kolejki asynchronicznej.

3.3.3 Warstwa danych (SQLite)

Ze względu na wymóg braku instalacji zewnętrznych serwerów bazodanowych, jako silnik bazy danych wybrano **SQLite**. Jest to bezserwerowa baza danych, przechowująca całą strukturę (użytkownicy, metadane plików, statusy zadań) w jednym pliku na dysku użytkownika. Zapewnia to zerową konfigurację i łatwość przenoszenia danych.

3.3.4 Warstwa przetwarzania asynchronicznego (Redis + Celery)

Generowanie skomplikowanych wizualizacji (np. detekcja społeczności w dużych grafach) jest realizowane asynchronicznie, aby nie blokować interfejsu użytkownika.

- **Redis (Embedded):** Pełni rolę brokera wiadomości. W wersji desktopowej wykorzystano przenośną wersję serwera Redis, uruchamianą lokalnie na porcie 6379.
- **Celery:** System kolejkowania zadań. Skonfigurowany do pracy z pulą wątków typu `solo` (ze względu na ograniczenia systemu Windows dotyczące funkcji `fork`), przetwarza zadania obliczeniowe w tle.

3.4 Komunikacja międzyprocesowa i zarządzanie

3.4.1 Skrypt nadzorczy (Supervisor)

Kluczowym elementem architektury jest autorski skrypt `backend_launcher.py`. Ponieważ Electron potrafi zarządzać bezpośrednio tylko jednym procesem potomnym, skrypt ten pełni rolę pośrednika (supervisora). Jego zadaniem jest:

1. Sekwencyjne uruchomienie serwera Redis.
2. Wykonanie niezbędnych migracji bazy danych SQLite.
3. Uruchomienie serwera aplikacji Django (WSGI/Development Server).
4. Uruchomienie procesu Celery Worker.
5. Monitorowanie stanu tych procesów i ich kaskadowe zamykanie w przypadku zakończenia działania aplikacji głównej.

3.4.2 Kanały komunikacji

Komunikacja między warstwą prezentacji (Angular) a warstwą logiki (Django) odbywa się poprzez protokół **HTTP REST**. Frontend traktuje backend jak zdalne API, wysyłając zapytania na adres lokalny. Autoryzacja realizowana jest przy pomocy tokenów JWT (JSON Web Token), co zapewnia bezpieczeństwo i bezstanowość sesji, nawet w środowisku lokalnym.

3.5 Topologia wdrożenia

Wdrożenie systemu odbywa się poprzez instalator (ForVis Desktop Setup 1.0.0.`exe`), który rozpakowuje kompletne środowisko uruchomieniowe na dysk użytkownika.

Struktura fizyczna wdrożenia (Topologia plików) wygląda następująco:

- **Katalog główny aplikacji:** Zawiera plik wykonywalny `ForVis.exe` (Electron) oraz biblioteki DLL.
- **resources/app/backend_dist/**: Skompilowany kod Python (zamrożony przy użyciu PyInstaller), zawierający interpreter Pythona, biblioteki Django/Celery oraz skrypt startowy.
- **resources/app/bin/**: Binaria serwera Redis (`redis-server.exe`).
- **%APPDATA%/ForVis/**: Katalog roboczy użytkownika, zawierający plik bazy danych (`db.sqlite3`), logi systemowe oraz przesłane pliki CNF.

Taka topologia zapewnia pełną izolację od systemu operacyjnego i pozwala na działanie wielu instancji aplikacji bez konfliktów portów czy zależności bibliotecznych.

Rozdział 4

Interfejsy i komunikacja

4.1 Wewnętrzny interfejs REST API

System ForVis Desktop udostępnia wewnętrzny interfejs REST API, z którego korzysta aplikacja Angular (Frontend) uruchamiana w środowisku Electron. Ze względu na architekturę desktopową, API działa lokalnie i jest odseparowane od sieci zewnętrznej.

4.1.1 Adres bazowy i konfiguracja

Serwer API nasłuchuje na interfejsie pętli zwrotnej pod adresem:

`http://127.0.0.1:8765/api/`

Komunikacja odbywa się z wykorzystaniem formatu JSON. Wszystkie endpointy (poza logowaniem i rejestracją) wymagają ważnego tokena JWT, przekazywanego w nagłówku HTTP:

```
Authorization: JWT <token>
```

4.2 Specyfikacja punktów końcowych (Endpoints)

4.2.1 Autoryzacja i zarządzanie sesją

Mechanizm uwierzytelniania oparty jest na tokenach JWT, co zapewnia bezstanowość komunikacji.

- **POST** `/api/auth/api-token-auth/`
  Logowanie użytkownika i pobranie tokena sesyjnego.
  - **Body:**

    ```
    {
        "username": "admin",
        "password": "password123"
    }
    ```

  - **Response (200 OK):**

```
{
    "token": "eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9..."
}
```

- **POST** `/api/auth/api-token-verify/`
  Weryfikacja ważności posiadanego tokena JWT.
- **POST** `/api/auth/register/`
  Rejestracja nowego użytkownika lokalnego (bez weryfikacji e-mail).

4.2.2 Zarządzanie plikami wejściowymi (SAT/CNF)

- **GET** `/api/profile/files/sat/`
  Pobranie listy plików SAT przypisanych do użytkownika.
- **PUT** `/api/profile/upload/sat/<filename>/`
  Przesłanie nowego pliku `.cnf`.
  - **Format:** `multipart/form-data` z polem `file`.
  - **Statusy:** `204` (OK), `400` (Błąd formatu).
- **DELETE** `/api/profile/file/sat/<id>/del/`
  Trwałe usunięcie pliku SAT.

4.2.3 Zarządzanie plikami Max-SAT

- **GET** `/api/profile/files/max-sat/`
  Pobranie listy plików Max-SAT.
- **PUT** `/api/profile/upload/maxsat/<filename>/`
  Przesłanie pliku Max-SAT (interpretacja wag klauzul).
- **DELETE** `/api/profile/file/max-sat/<id>/del/`
  Usunięcie pliku Max-SAT.

4.2.4 Sterowanie procesem wizualizacji

Proces generowania wizualizacji jest asynchroniczny (Celery).

- **POST** `/api/profile/run/<format>/<sat_id>/`
  Zlecenie zadania generowania wizualizacji.
  - **Response:**

    ```
    {
        "task_id": "e2fa31ef-917c-4f3f-bbf9-72c03d8894c3"
    }
    ```

- **GET** `/api/profile/task/<task_id>/progress/`
  Polling statusu zadania.
  - **Response:**

```
{
    "state": "PENDING",
    "progress": "0%"
}
```

4.2.5 Dostęp do danych wynikowych

- **GET** `/api/profile/visualizations`
  Lista wygenerowanych wizualizacji wraz z ich statusem.
- **GET** `/api/profile/file/sat/<id>/<format>/`
  Pobranie pełnych danych wizualizacji w formacie JSON.
- **GET** `/api/profile/download/<id>/`
  Pobranie wyeksportowanych plików graficznych.

4.3 Komunikacja międzyprocesowa (IPC)

W systemie ForVis Desktop rolę orkiestratora pełni proces główny Electrona (Main Process). Komunikacja z podsystemem backendowym (Python) nie odbywa się poprzez standardowe kanały IPC Electrona, lecz poprzez zarządzanie standardowymi strumieniami wejścia/wyjścia (stdio).

4.3.1 Uruchamianie i nadzór (Spawn)

Podczas startu aplikacji, Electron wykorzystuje natywną funkcję Node.js `child_process.spawn()` do uruchomienia pliku wykonywalnego `backend_launcher.exe`.

- **Argumenty startowe:** Ścieżka do bazy danych oraz port serwera.
- **Separacja środowiska:** Proces potomny dziedziczy zmienne środowiskowe, ale działa w odrębnej przestrzeni pamięci.

4.3.2 Przekierowanie strumieni (Logging)

Standardowe wyjście (`stdout`) oraz wyjście błędów (`stderr`) procesu Python są przechwytywane przez Electrona i zapisywane do pliku logów w katalogu użytkownika:

$$\%APPDATA\% \setminus ForVis \setminus logs \setminus main.log$$

Umożliwia to debugowanie błędów backendu (np. wyjątków Django) z poziomu aplikacji desktopowej.

4.3.3 Sygnały sterujące (Graceful Shutdown)

Zamknięcie okna aplikacji wysyła sygnał zakończenia do procesu głównego. Electron, przed zamknięciem, wysyła sygnał `SIGTERM` (lub `taskkill` na Windows) do procesu Supervisora. Skrypt `backend_launcher.py` przechwytuje ten sygnał i bezpiecznie zamyka podprocesy (Redis, Celery, Django), zapobiegając uszkodzeniu pliku bazy danych SQLite.

4.4 Formaty wymiany danych

Podstawowym formatem wymiany danych wizualizacyjnych jest JSON. Struktura ta została zoptymalizowana pod kątem renderowania grafów w bibliotece D3.js (używanej przez Angular).

Przykładowa struktura odpowiedzi dla Grafu Interakcji:

```
{
  "nodes": [
    {"id": 1, "group": "variable", "label": "x1"},
    {"id": 2, "group": "variable", "label": "x2"}
  ],
  "links": [
    {"source": 1, "target": 2, "value": 1}
  ],
  "metadata": {
    "num_vars": 2,
    "num_clauses": 1
  }
}
```

Rozdział 5

Opis modułów wizualizacyjnych

W ramach dotychczasowych prac nad systemem zaimplementowano jedenaście zróżnicowanych metod wizualizacji danych logicznych. Zostały one oparte na dwóch kluczowych bibliotekach ekosystemu NPM, które umożliwiły prezentację formuł zarówno w ujęciu statystycznym, jak i strukturalnym:

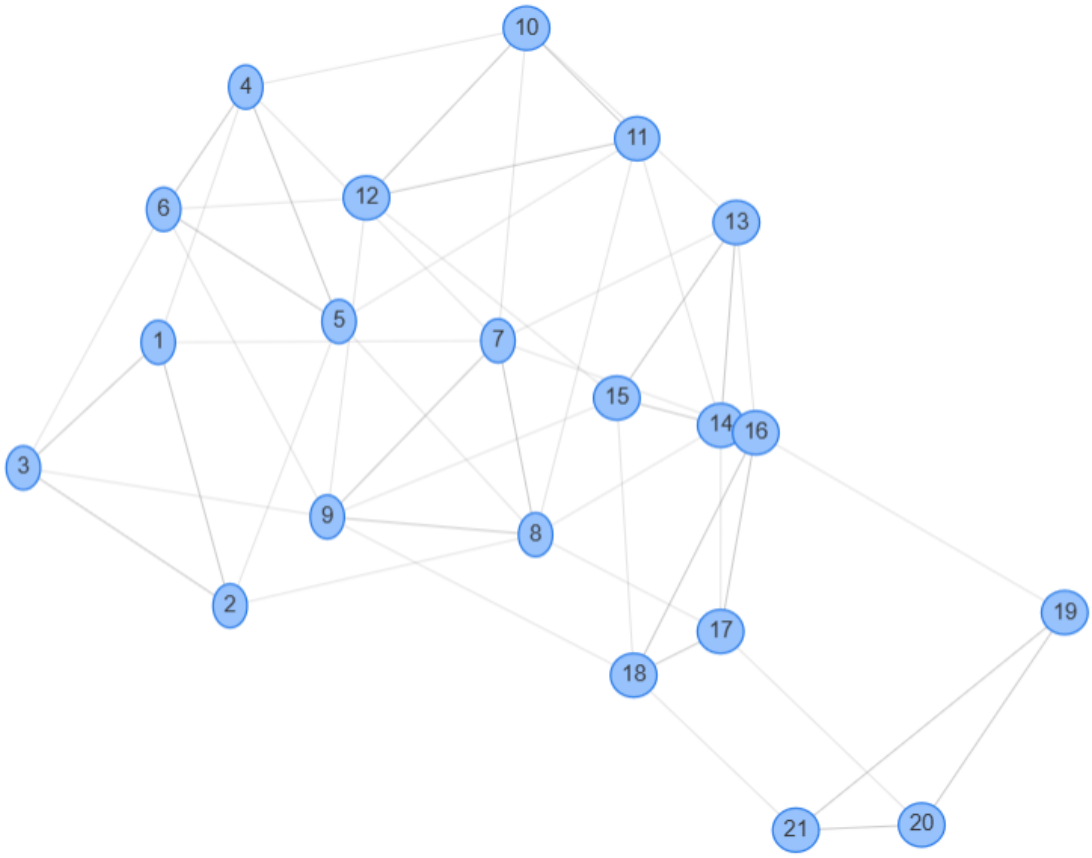
- **Chart.js** – wykorzystana głównie do prezentacji danych ilościowych oraz gęstości występowania zmiennych. W oparciu o ten moduł stworzono:
  - *Distribution Chart* – wykres rozkładu zmiennych,
  - *Heatmap* – mapę intensywności relacji między elementami formuły.
- **Vis-network** – zaawansowane narzędzie do wizualizacji grafów i sieci, użyte do przedstawienia topologii problemów SAT oraz procesów ich rozwiązywania. Moduł ten obsługuje:
  - *Interaction Graph* oraz *2-clauses Interaction Graph*,
  - *Factor Graph* oraz *Direct Graphical Model*,
  - *Resolution Graph* – graf rezolucji,
  - *Tree* oraz *Cluster* – struktury hierarchiczne i skupiska,
  - *DPLL Solver* – wizualizację przebiegu algorytmu przeszukiwania.

Oprócz wymienionych metod graficznych, zaimplementowano również moduł *Matrix Visualization*. Ze względu na specyfikę danych oraz potrzebę zachowania maksymalnej czytelności przy dużych zbiorach zależności, do jego budowy wykorzystano strukturę klasycznej macierzy w formie interaktywnej tabeli.

Do wykonania przykładowych wizualizacji został użyty plik o nazwie *coloring.cnf*, który zawiera 21 zmiennych oraz 58 klauzul.

5.1 Interaction Graph

Graf interakcji to nieskierowany graf, w którym wierzchołki odpowiadają literałom, a krawędzie reprezentują współwystępowanie literałów w tej samej klauzuli.



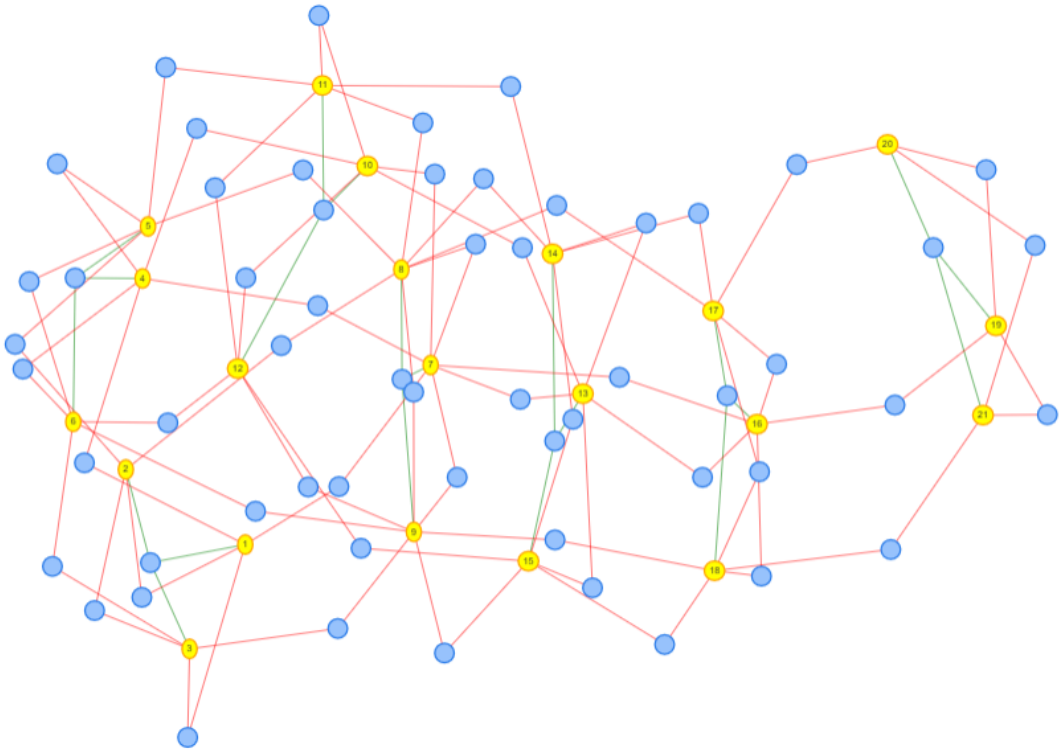
Rysunek 5.1: Interaction graph

Implementacja:

- Węzły reprezentują literały.
- Krawędzie łączą literały współwystępujące w jednej klauzuli.
- Duplikaty krawędzi są eliminowane.
- Algorytm iteruje po wszystkich klauzulach i łączy pary literałów, tworząc strukturę grafu uwypuklającą interakcje między zmiennymi

5.2 Factor Graph

Graf czynnikowy stanowi rozszerzenie grafu interakcji poprzez dodanie węzłów reprezentujących klauzule. Krawędzie łączą literały z klauzulami, w których występują, co zapewnia bardziej szczegółową reprezentację.



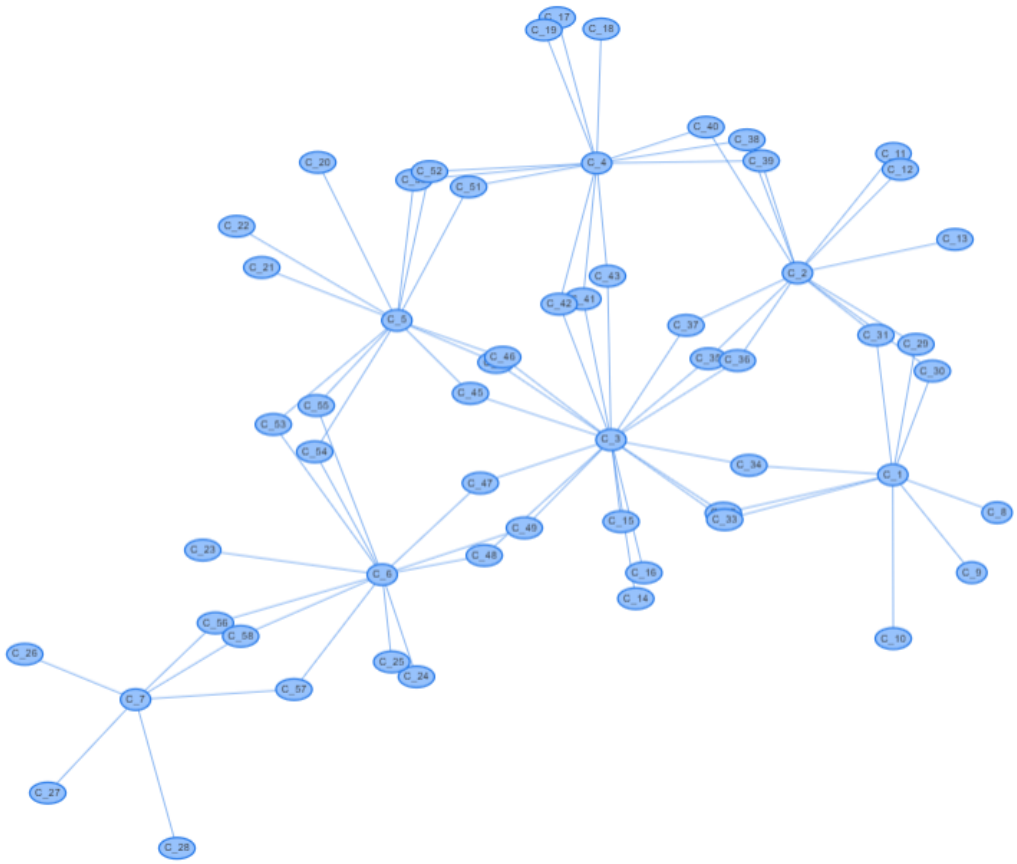
Rysunek 5.2: Factor graph

Implementacja:

- Węzły reprezentują zarówno literały, jak i klauzule.
- Krawędzie łączą literały z odpowiadającymi im klauzulami.
- Kolor krawędzi wskazuje typ literału: zielony dla dodatnich, czerwony dla ujemnych.
- Algorytm analizuje klauzule i przypisuje literały do właściwych węzłów klauzul.

5.3 Resolution graph

Graf rozdzielczości przedstawia relacje między klauzulami. Węzły odpowiadają klauzulom, a krawędzie są dodawane, gdy klauzule zawierają wspólne literały komplementarne.



Rysunek 5.3: Resolution graph

Implementacja:

- Węzły reprezentują klauzule.
- Krawędzie są tworzone, gdy w jednej klauzuli występuje literał, a w innej jego negacja.
- Graf wizualizuje zależności logiczne oraz etapy rozdzielczości.
- Eliminacja zbędnych krawędzi zwiększa przejrzystość wizualizacji.

5.4 Tree visualization

Wizualizacja drzewiasta przedstawia hierarchiczną strukturę formuł, ukazując zależności między klauzulami i zmiennymi.

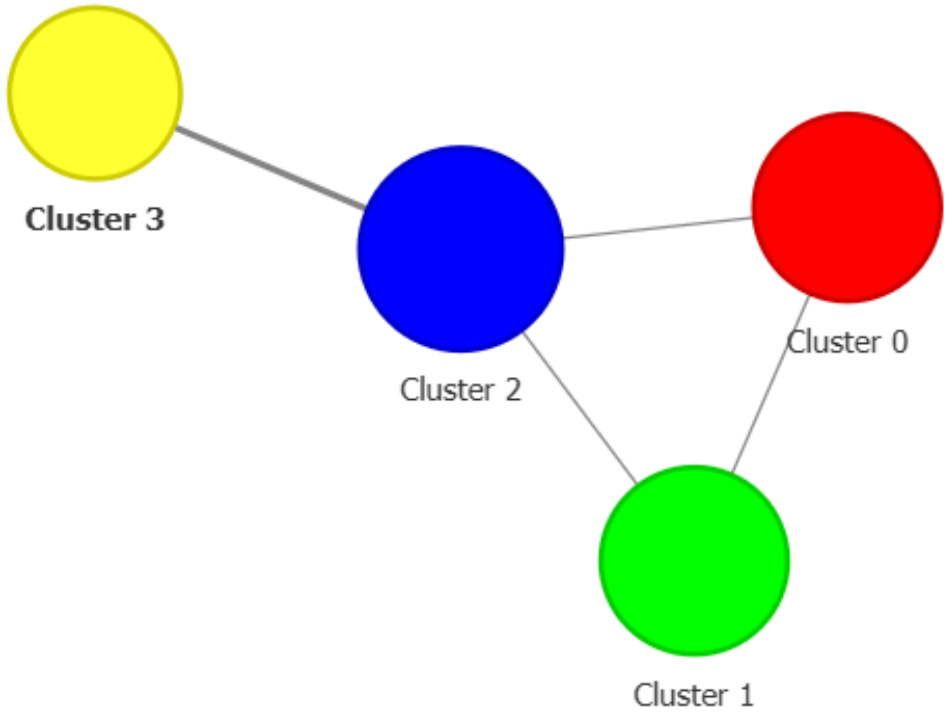
Rysunek 5.4: Tree visualization

Implementacja:

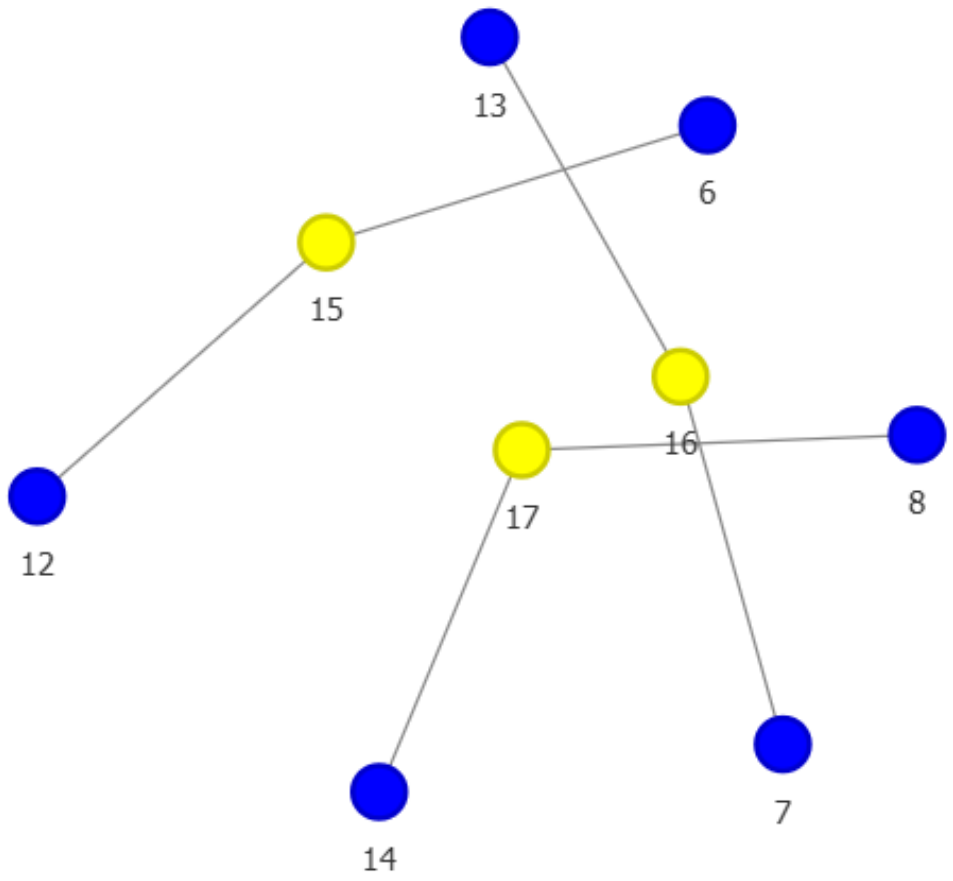
- Węzły reprezentują klauzule, a krawędzie reprezentują zależności logiczne.
- Hierarchiczny układ organizuje drzewo dla lepszej czytelności.
- Algorytm analizuje klauzule i buduje strukturę drzewa na podstawie zależności.

5.5 Cluster Visuzalization

Wizualizacja klastrów grupuje powiązane literały i klauzule, ukazując modułową strukturę formuły.



Rysunek 5.5: Cluster visualization



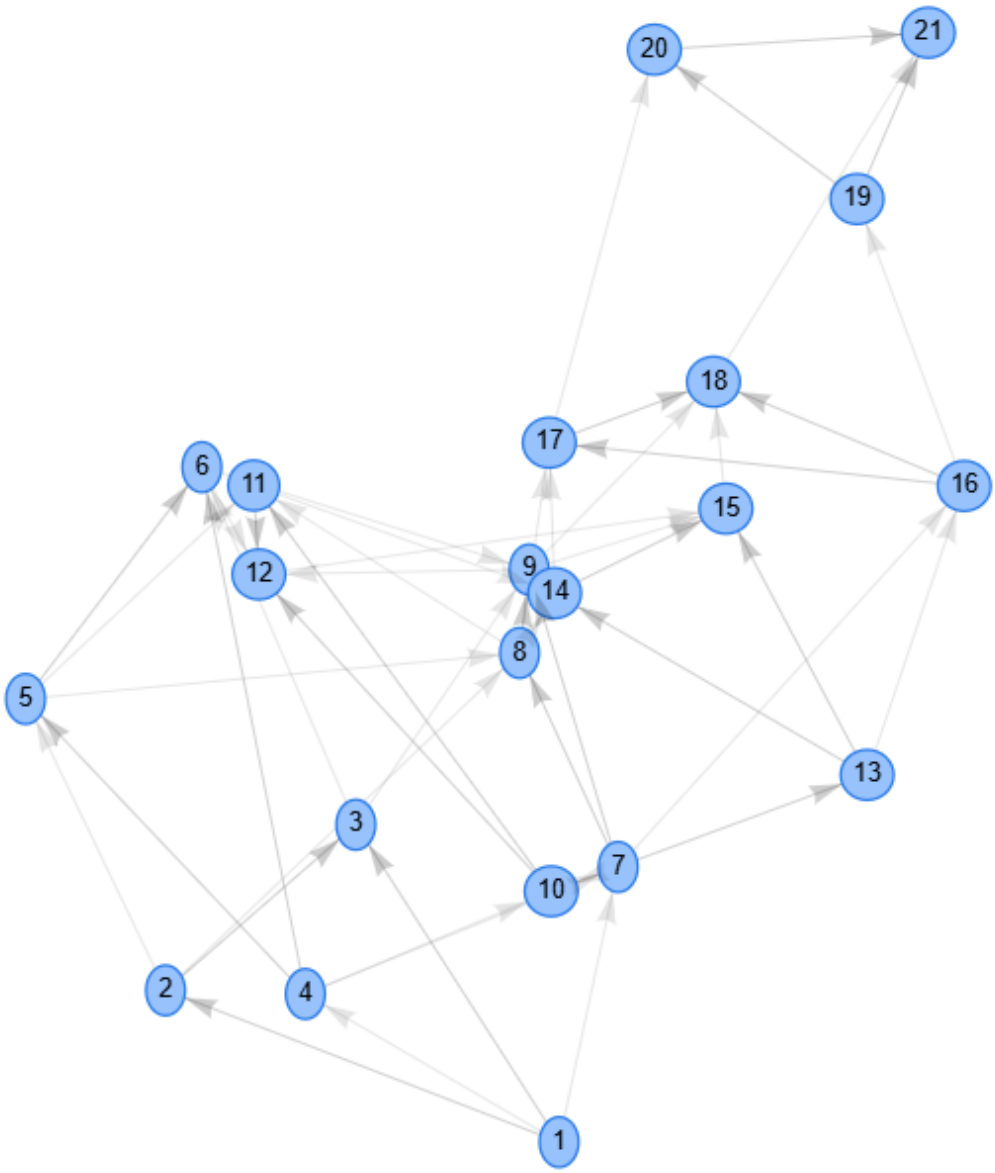
Rysunek 5.6: Cluster visualization - widok powiększony

Implementacja:

- Algorytmy wykrywania społeczności identyfikują klastry.
- Węzły są kolorowane zgodnie z przynależnością do klastrów.
- Minimalizuje się liczbę krawędzi między klastrami w celu uwydatnienia modułowości.

5.6 Direct Graphical Model

Model graficzny przedstawia klauzule i literały jako węzły połączone krawędziami skierowanymi, wskazującymi logiczne implikacje.



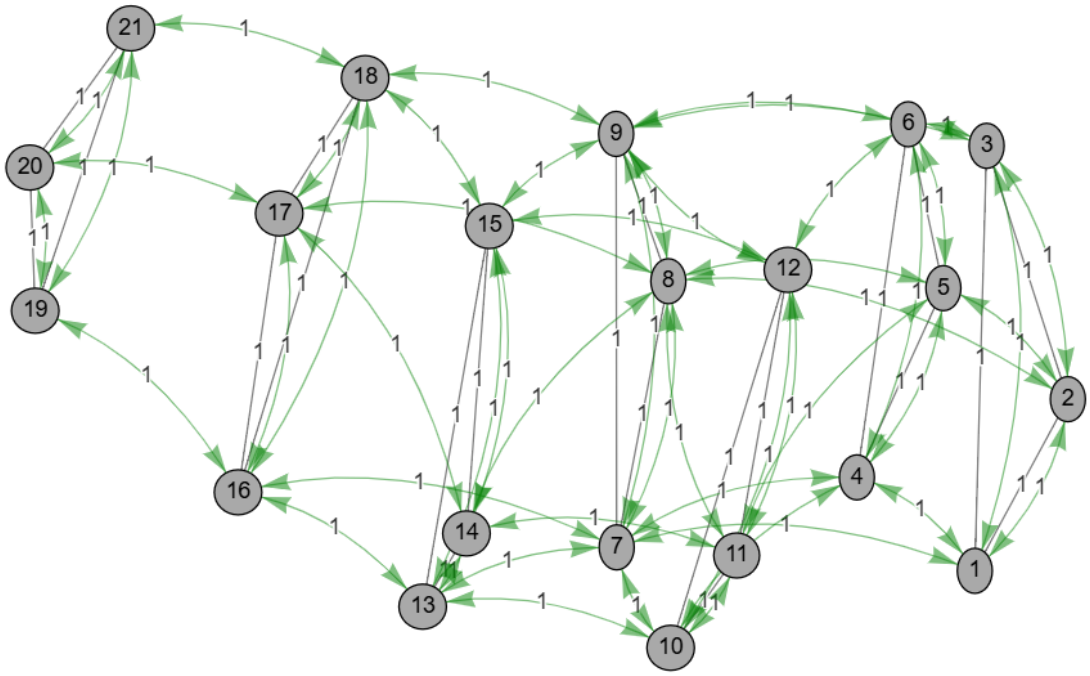
Rysunek 5.7: Direct Graphical Model

Implementacja:

- Węzły reprezentują literały i klauzule.
- Skierowane krawędzie wskazują implikacje lub zależności logiczne.
- Algorytm wykorzystuje układy oparte na modelach fizycznych w celu poprawy przejrzystości.

5.7 2-Clauses Interaction Graph

Wizualizacja koncentruje się na relacjach w formułach CNF składających się z klauzul dwuelementowych, uwypuklając ich interakcje.



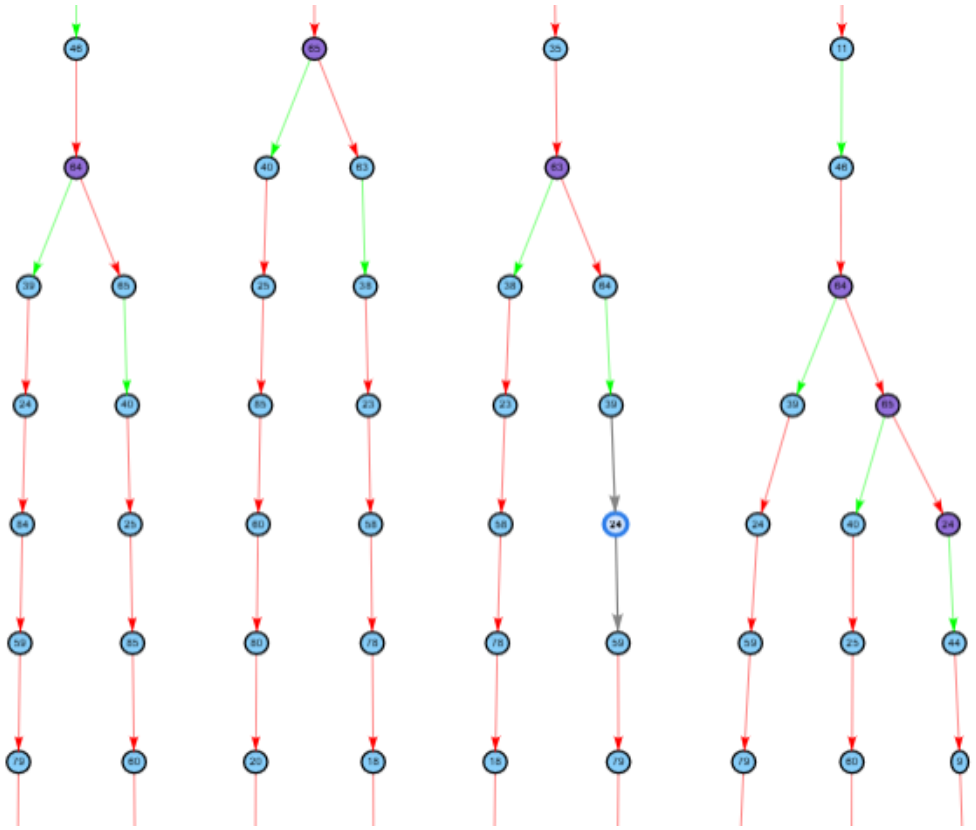
Rysunek 5.8: 2-Clauses Interaction Graph

Implementacja:

- Węzły reprezentują literały.
- Krawędzie łączą literały w tej samej klauzuli dwuelementowej.
- Style i kolory krawędzi rozróżniają typy interakcji (dodatni-dodatni, ujemny-ujemny, mieszany).

5.8 DPLL Solver Visualization

Wizualizacja ta ilustruje proces podejmowania decyzji w algorytmie Davisa-Putnama-Logemanna-Lovelanda (DPLL), w tym przypisania zmiennych i wykrywania konfliktów.



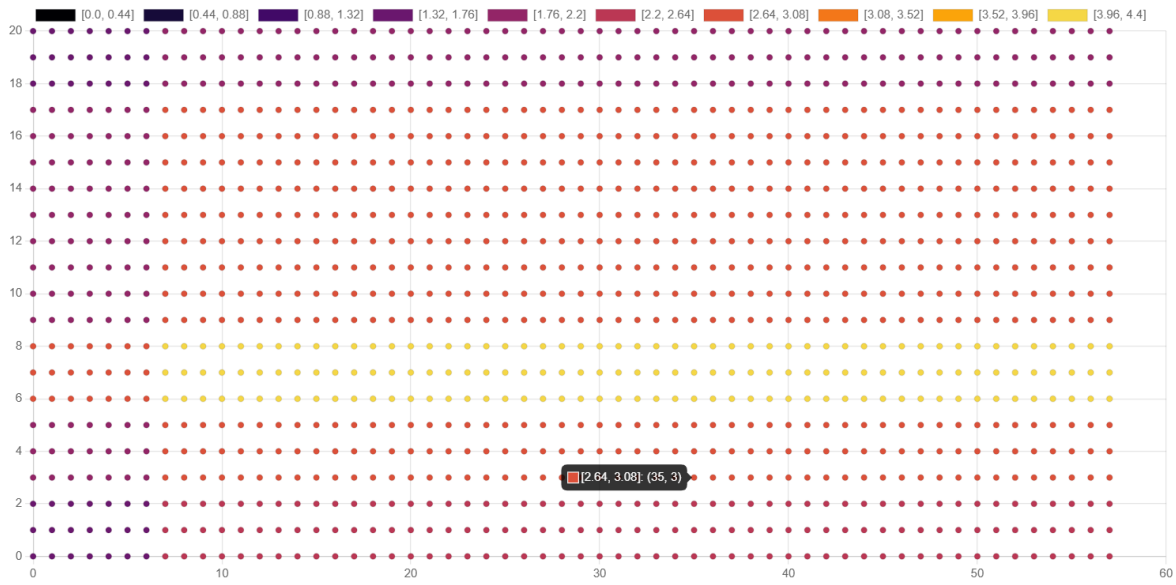
Rysunek 5.9: Fragment DPLL Solver Visualization

Implementacja:

- Węzły reprezentują decyzje, konflikty lub stany końcowe (SAT/UNSAT).
- Skierowane krawędzie przedstawiają przebieg drzewa decyzyjnego.
- Heurystyki, takie jak Dynamic Largest Individual Sum (DLIS), Jeroslow-Wang i Maximum number of Occurences in the Minimum length clauses (MOMs), kierują wyborem węzłów decyzyjnych.
- Kolory odróżniają typy i stany węzłów.

5.9 Heatmap Visualization

Mapa ciepła przedstawia intensywność relacji między klauzulami a zmiennymi, wizualizując ich częstotliwość.



Rysunek 5.10: Heatmap visualization

Implementacja:

- Macierz odwzorowuje klauzule (wiersze) i literały (kolumny).
- Intensywność koloru odzwierciedla częstotliwość zmiennych w klauzulach.
- Dane są skalowane w celu poprawy przejrzystości, a wartości odstające wyróżniane innymi kolorami.

5.10 Matrix Visualization

Macierz zależności wizualizuje strukturę powiązań między zmiennymi w formule, wskazując na siłę ich współwystępowania w tych samych klauzulach z uwzględnieniem polaryzacji (pozytywnej i negatywnej).

#	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20
0	x	x	x	x			x														
1	x	x	x		x			x													
2	x	x	x			x			x												
3	x			x	x	x	x			x											
4		x		x	x	x		x			x										
5			x	x	x	x			x			x									
6	x			x			x	x	x	x			x			x					
7		x			x		x	x	x		x			x			x				
8			x			x	x	x	x			x			x			x			
9				x			x			x	x	x	x								
10					x			x		x	x	x		x							
11						x			x	x	x	x			x						
12							x			x			x	x	x	x					
13								x			x		x	x	x		x				
14									x			x	x	x	x			x			
15							x						x			x	x	x	x		
16								x						x		x	x	x		x	
17									x						x	x	x	x			x
18																x			x	x	x
19																	x		x	x	x
20																		x	x	x	x

Rysunek 5.11: Matrix Visualization

Implementacja:

- Macierz kwadratowa, gdzie zarówno wiersze, jak i kolumny reprezentują zmienne ($N \times N$).
- Każda komórka przechowuje licznik wspólnych wystąpień zmiennej A i B w ramach jednej klauzuli.
- Rozróżniane są dwa rodzaje powiązań (pozytywne i negatywne) w zależności od znaku literału w formule DIMACS.
- Algorytm wspiera przetwarzanie wsadowe (checkpointing), umożliwiając analizę bardzo dużych plików CNF poprzez zapisywanie stanu pośredniego (snapshot) i wznawianie obliczeń.

Rozdział 6

Mechanizmy przetwarzania i sterowania obliczeniami

Implementacja algorytmów wizualizacji problemów NP-trudnych (jakim jest SAT) wiąże się z ryzykiem długotrwałych obliczeń. W wersji desktopowej systemu, priorytetem stało się zapewnienie responsywności interfejsu oraz danie użytkownikowi kontroli nad procesami działającymi w tle.

Rozdział ten opisuje architekturę przetwarzania asynchronicznego oraz autorski mechanizm wywłaszczania zadań (Stop/Resume), zaprojektowany z uwzględnieniem specyficznych ograniczeń środowiska Windows.

6.1 Środowisko wykonawcze Celery na Windows

System wykorzystuje bibliotekę **Celery** w wersji 4.x do kolejkowania zadań. W standardowych środowiskach serwerowych (Linux), Celery wykorzystuje mechanizm `fork()` do tworzenia puli procesów roboczych (*prefork pool*). Systemy z rodziny Windows nie obsługują funkcji `fork()`, co wymusiło zastosowanie alternatywnej strategii wykonawczej.

6.1.1 Model wykonawczy `solo`

Wdrożona instancja Celery została skonfigurowana w trybie puli `solo`. Oznacza to, że proces *worker* przetwarza zadania sekwencyjnie w tym samym wątku, w którym działa pętla główna procesu.

- **Zaleta:** Eliminacja problemów z serializacją danych między procesami (zgodność z `PyInstaller`).
- **Wyzwanie:** Zadania w trybie `solo` są domyślnie blokujące. Jeśli zadanie obliczeniowe wejdzie w pętlę, nie można go przerwać z zewnątrz bez ubijania całego procesu workera.

Powyższe uwarunkowanie wymusiło implementację mechanizmu kooperacyjnego wywłaszczania zadań bezpośrednio w kodzie algorytmów wizualizacji.

6.2 Mechanizm Stop/Resume (Application-Level Preemption)

Zamiast polegać na sygnałach systemowych (które są niestabilne w opisywanym stosie technologicznym), zaimplementowano mechanizm przerywania na poziomie aplikacji.

6.2.1 Strategia Time-Based Throttling

Aby umożliwić wstrzymanie (Pauza) lub anulowanie (Stop) zadania, algorytmy wizualizacji zostały wzbogacone o punkty kontrolne (checkpoints). W punktach tych system sprawdza status zadania w bazie danych SQLite.

Aby zminimalizować narzut operacji wejścia-wyjścia (I/O Overhead) związany z ciągłym odpytywaniem bazy danych, zastosowano strategię dławienia czasowego (*Time-Based Throttling*). Sprawdzenie flagi przerwania następuje nie częściej niż co 1 sekundę.

Pseudokod mechanizmu kontrolnego:

```
def check_interruption(task_id, last_check_time):
    current_time = time.time()
    # Sprawdzaj bazę nie częściej niż co 1000ms
    if current_time - last_check_time < 1.0:
        return False, last_check_time

    status = DB.get_status(task_id)

    if status == 'PAUSED':
        wait_loop() # Aktywne oczekiwanie na wznowienie
    elif status == 'REVOKED':
        raise TaskRevokedError() # Przerwanie obliczeń

    return True, current_time
```

Dzięki temu rozwiązaniu, narzut czasowy na obsługę mechanizmu Stop/Resume jest pomijalny dla użytkownika, a system zachowuje pełną responsywność.

6.3 Kategoryzacja zadań wizualizacyjnych

Ze względu na specyfikę obliczeniową, wizualizacje podzielono na trzy kategorie obsługi przerwań.

6.3.1 Kategoria 1: Zadania standardowe (Przerywalne)

Dotyczy wizualizacji o złożoności liniowej lub wielomianowej względem rozmiaru wejścia (np. *Heatmap*, *Tree Visualization*, *Cluster Graph*). W tych modułach punkty kontrolne umieszczono w głównych pętlach iterujących po klauzulach lub zmiennych. Mechanizm pauzowania działa tu z pełną skutecznością.

6.3.2 Kategoria 2: Zadania natychmiastowe (Instant)

Dotyczy prostych grafów (np. *Interaction Graph* dla małych plików). Czas generowania tych wizualizacji wynosi zazwyczaj poniżej 500 ms. Mechanizm sprawdzania statusu jest w nich zaimplementowany jako "bezpiecznik"(failsafe), który aktywuje się tylko w przypadku wczytania ekstremalnie dużych plików wejściowych.

6.3.3 Kategoria 3: Wyjątek algorytmiczny (Solver DPLL)

Wizualizacja algorytmu DPLL stanowi wyjątek, w którym celowo zablokowano możliwość pauzowania w trakcie krytycznych faz obliczeń.

Uzasadnienie techniczne: Algorytm wykorzystuje heurystykę MOMS (*Maximum Occurrence in clauses of Minimum Size*), która opiera się na sortowaniu list klauzul. Natywna metoda Pythona `list.sort()` jest operacją atomową realizowaną w języku C.

- Próba zastąpienia natywnego sortowania własną implementacją z obsługą przerwań spowodowała wzrost czasu obliczeń o ok. **400%**.
- Uznano, że degradacja wydajności jest zbyt wysoka względem korzyści z funkcjonalności pauzy.

W przypadku DPLL system wyświetla użytkownikowi ostrzeżenie przed uruchomieniem zadania, informując o braku możliwości jego wstrzymania.

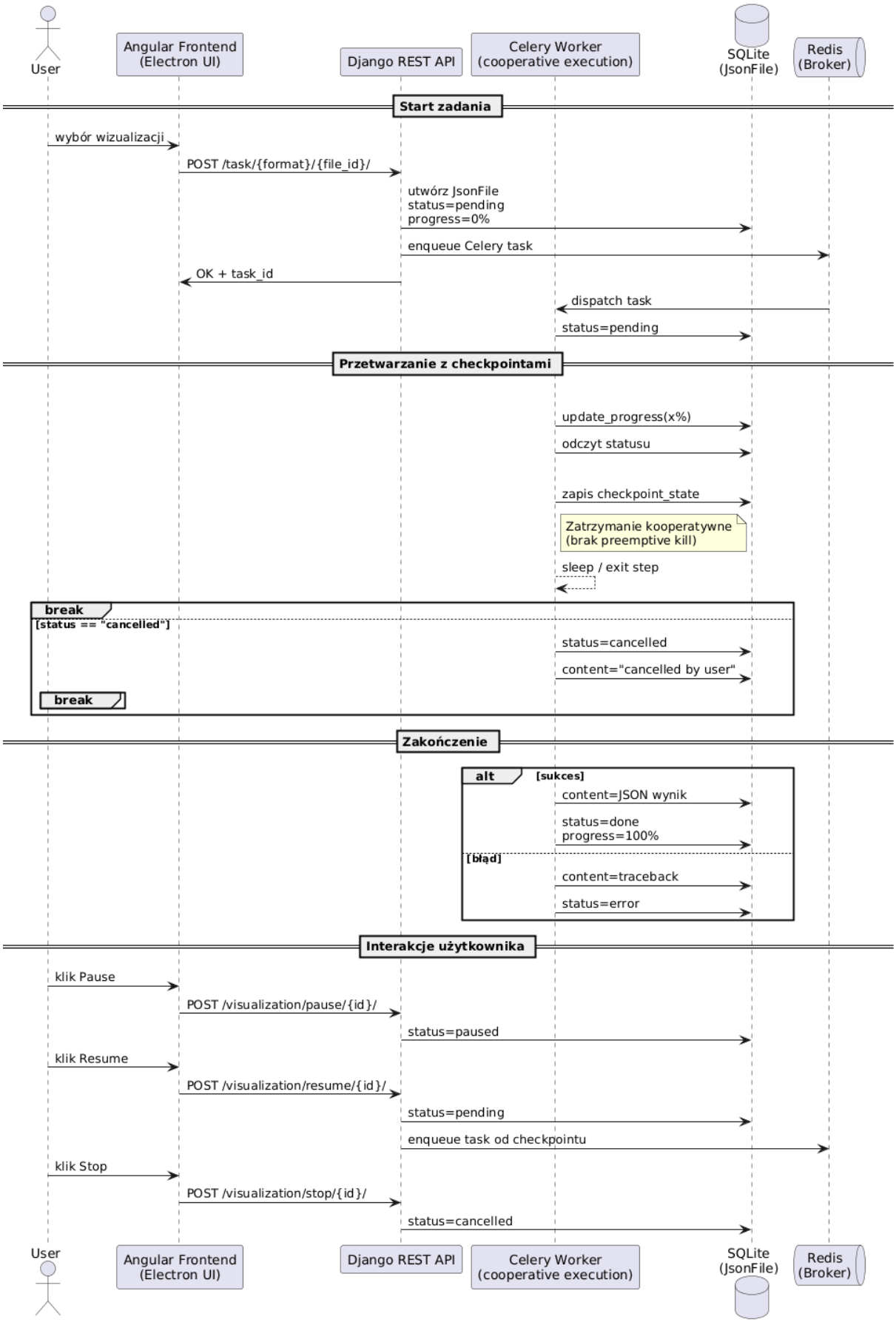
6.4 Obsługa błędów obliczeniowych

Ze względu na izolowane środowisko, system implementuje globalny mechanizm przechwytywania wyjątków (Global Exception Handler) wewnątrz workera Celery.

- Każdy błąd (np. `MemoryError` przy zbyt dużej formule) jest przechwytywany.
- Status zadania w bazie zmieniany jest na `FAILURE`.
- Szczegóły błędu (Stack Trace) są zapisywane do logów w `%APPDATA%`, a użytkownik otrzymuje czytelny komunikat w GUI zamiast "zamrożenia"aplikacji.

6.5 Przepływ

Kompletny przepływ sterowania, obejmujący inicjalizację zadania, cykliczne sprawdzanie punktów kontrolnych przez workera oraz obsługę żądań użytkownika (Pauza/Wznowienie), przedstawiono na Rysunku 6.1.



Rysunek 6.1: Szczegółowy diagram sekwencji cyklu życia zadania asynchronicznego z uwzględnieniem mechanizmu kooperacyjnego wywłaszczania.

Rozdział 7

Bezpieczeństwo, niezawodność i diagnostyka systemu

W systemie zaimplementowano szereg mechanizmów chroniących przed awariami, zapewniających integralność przetwarzanych danych oraz umożliwiających diagnostykę powdrożeniową. Ze względu na lokalny charakter aplikacji desktopowej oraz brak zdalnego dostępu do backendu, zagadnienia bezpieczeństwa sieciowego (np. TLS, uwierzytelnianie transportowe) nie stanowiły głównego obszaru projektowego.

7.1 Mechanizmy zapewnienia integralności i bezpieczeństwa danych

Ze względu na lokalny charakter wdrożenia (aplikacja desktopowa), główny wektor zagrożeń stanowi integralność plików wejściowych oraz bezpieczne przetwarzanie struktur danych. W warstwie backendu zastosowano następujące zabezpieczenia:

7.1.1 Sanityzacja danych wejściowych (Safe Evaluation)

System przechowuje stany pośrednie wizualizacji w formacie JSON, gdzie klucze słowników (np. krawędzie grafu) są często reprezentowane jako krotki (ang. *tuples*). Podczas deserializacji tych danych unika się stosowania niebezpiecznej funkcji `eval()`, która mogłaby posłużyć do wstrzyknięcia złośliwego kodu (Code Injection).

Zamiast tego, w module `tasks.py` zaimplementowano funkcję `restore_tuple_keys`, wykorzystującą bezpieczny parser drzewa składniowego:

```
import ast

def restore_tuple_keys(d):
    # ...
    try:
        # Bezpieczna ewaluacja struktury danych bez wykonywania kodu
        converted_key = ast.literal_eval(k)
        # ...
```

Wykorzystanie biblioteki `ast` (Abstract Syntax Tree) gwarantuje, że przetwarzane są wyłącznie literały Pythona, co eliminuje ryzyko wykonania nieautoryzowanych operacji systemowych.

7.1.2 Walidacja formatów DIMACS/CNF

Parsery plików wejściowych (zaimplementowane dla każdego typu wizualizacji) działają w trybie defensywnym. Pętle przetwarzające (np. w funkcji `create_sat_vis_heatmap`) ignorują linie, które nie spełniają ścisłych kryteriów formatowania (np. linie komentarzy, puste linie, błędne nagłówki). Dzięki temu uszkodzony lub niepełny plik wejściowy nie powoduje przerwania działania całej aplikacji, a jedynie pominięcie błędnych danych.

7.2 System obsługi błędów krytycznych (Fault Tolerance)

W celu zapewnienia ciągłości działania aplikacji (nawet w przypadku wystąpienia nieprzewidzianych błędów w algorytmach wizualizacyjnych), wdrożono globalny mechanizm przechwytywania wyjątków na poziomie zadania (Task Level).

7.2.1 Kapsułkowanie procesu wykonawczego

Główna funkcja sterująca `create_json` została objęta blokiem `try-except`, który pełni rolę "ostatniej deski ratunku"(ang. *safety net*).

- **Scenariusz błędu:** W przypadku wystąpienia nieobsłużonego wyjątku (np. `MemoryError`, `ValueError`), worker Celery nie ulega awarii.
- **Propagacja błędu:** Wyjątek jest przechwytywany, a jego treść (`traceback`) jest serializowana do formatu JSON i zapisywana w bazie danych w polu `content` obiektu `JsonFile`.
- **Informacja zwrotna:** Status zadania w bazie danych zmieniany jest na `'error'`. Dzięki temu interfejs użytkownika (Frontend) może wyświetlić stosowny komunikat błędu zamiast nieskończonego paska ładowania.

7.3 Logowanie strumieniowe (Stdout/Stderr)

Ze względu na architekturę procesu uruchamianego przez PyInstaller, standardowe wyjście (`stdout`) procesu backendu jest przekierowywane do konsoli nadzorcy (Supervisor). W kodzie zadań zastosowano funkcję `print(..., flush=True)`, co wymusza natychmiastowe opróżnienie bufora wyjścia. Pozwala to na śledzenie postępu przetwarzania (np. `"SAT_VIS_INTERACTION STARTED"`) w czasie rzeczywistym, co jest kluczowe podczas debugowania w środowisku produkcyjnym Windows.

Rozdział 8

Instalacja, konfiguracja i proces wdrożenia

Zgodnie z wytycznymi normy ISO/IEC/IEEE 26511, niniejszy rozdział definiuje procedury uruchomieniowe. Ze względu na dualny charakter projektu, proces ten podzielono na dwie odrębne ścieżki: bezobsługową instalację dla użytkownika końcowego oraz konfigurację środowiska dla deweloperów.

8.1 Wdrożenie produkcyjne (Użytkownik końcowy)

Głównym celem procesu dystrybucji było całkowite ukrycie złożoności technologicznej przed użytkownikiem. Aplikacja dostarczana jest w formie samowystarczalnej paczki binarnej (ang. *Standalone Executable*), która zawiera w sobie wszystkie niezbędne komponenty.

8.1.1 Wymagania dla użytkownika końcowego

Dzięki zastosowaniu techniki "zamrażania"kodu (PyInstaller) oraz dołączeniu przenośnych wersji serwerów, wymagania po stronie użytkownika zostały zredukowane do minimum:

- **System operacyjny:** Windows 10 lub Windows 11 (x64).
- **Środowisko uruchomieniowe: BRAK**. Użytkownik nie musi instalować Pythona, Node.js, Redisa ani żadnych bibliotek zewnętrznych. Wszystkie zależności są zaszyte wewnątrz pliku wykonywalnego.
- **Uprawnienia:** Standardowe konto użytkownika (prawa administratora nie są wymagane do uruchomienia wersji Portable).

8.1.2 Procedura uruchomienia

Proces instalacji został maksymalnie uproszczony, realizując paradygmat "Click & Run":

1. **Krok 1:** Pobranie instalatora `ForVis_Setup.exe` lub archiwum ZIP.
2. **Krok 2:** Rozpakowanie archiwum (w przypadku wersji ZIP).

3. **Krok 3:** Uruchomienie pliku `ForVis.exe`.

System automatycznie, w sposób transparentny dla użytkownika, powołuje do życia w tle lokalne instancje bazy danych, brokera wiadomości oraz silnika obliczeniowego. Po zamknięciu okna aplikacji, wszystkie te procesy są automatycznie sprzątane.

8.2 Konfiguracja środowiska deweloperskiego (Dla programistów)

Poniższa sekcja przeznaczona jest wyłącznie dla osób chcących modyfikować kod źródłowy systemu. Wymaga ona ręcznej instalacji pełnego stosu technologicznego.

8.2.1 Wymagania programistyczne

Aby uruchomić system ze źródeł, stacja robocza musi posiadać:

- Interpreter **Python 3.6** (wymóg ścisły ze względu na kompatybilność wsteczną bibliotek `tensorflow` i `celery 4.x`).
- Platformę **Node.js** (wersja LTS) do budowania warstwy prezentacji.
- Serwer **Redis** (uruchomiony jako usługa systemowa lub proces w tle).

8.2.2 Instalacja zależności

Backend (Python)

```
# Tworzenie wirtualnego srodowiska
python -m venv venv
.\venv\Scripts\activate

# Instalacja zaleznosci
pip install -r requirements.txt

# Inicjalizacja bazy danych
python manage.py migrate
```

Frontend (Angular)

```
cd frontend
npm install
npm run build:prod  # Kompilacja do plikow statycznych
```

8.3 Proces budowania wersji dystrybucyjnej

Aby przekształcić środowisko deweloperskie w gotowy plik `.exe` dla użytkownika końcowego, wykorzystywany jest skrypt automatyzacji oparty na `PyInstaller`.

Plik specyfikacji `backend.spec` odpowiada za:

1. Wykrycie i dołączenie ukrytych importów (dynamicznie ładowanych bibliotek Django i Celery).
2. Dołączenie zasobów statycznych (skompilowany Frontend Angulara).
3. Dołączenie binariów pomocniczych (np. `satelite.exe` do minimalizacji).

Komenda budująca:

```
pyinstaller backend.spec --noconfirm --clean
```

Rozdział 9

Testowanie i walidacja systemu

Proces weryfikacji systemu został podzielony na trzy etapy: testy jednostkowe kluczowych algorytmów, testy funkcjonalne (manualne) interfejsu użytkownika oraz testy wydajnościowe mechanizmów współbieżnych.

9.1 Testy jednostkowe i integracyjne (Backend)

Ze względu na skomplikowaną logikę parsowania plików DIMACS (SAT/CNF) oraz MaxSAT, kluczowym elementem było zapewnienie poprawności interpretacji danych wejściowych.

9.1.1 Weryfikacja parserów

Zaimplementowano testy sprawdzające poprawność funkcji parsujących w module `tasks.py` dla różnych formatów wejściowych:

- **Format CNF:** Weryfikacja poprawności odczytu klauzul, obsługi komentarzy oraz linii nagłówkowych (`p cnf ...`).
- **Format WCNF (MaxSAT):** Sprawdzenie poprawności wyodrębniania wag klauzul (pierwsza liczba w linii) dla wizualizacji *MaxSAT Resolution* i *MaxSAT Factor Graph*.
- **Obsługa błędów:** Testy odporności na uszkodzone pliki (np. brak nagłówka, puste linie, znaki nieliczbowe). System poprawnie ignoruje błędne linie zamiast przerywać działanie (co zapewnia funkcja `check_interruption` i bloki `try-except`).

9.1.2 Weryfikacja struktur grafowych

Dla wizualizacji opartych na grafach (np. *Interaction Graph*, *Direct Graphical Model*) przeprowadzono testy weryfikujące poprawność budowania krawędzi:

- Sprawdzono, czy dla klauzul o długości $N > 2$ generowana jest odpowiednia liczba krawędzi (zgodnie ze wzorem na kombinacje $N2$).
- Zweryfikowano poprawność sortowania wierzchołków w krawędziach, aby uniknąć duplikatów (np. krawędź 1-2 i 2-1 jest traktowana jako ta sama).

9.2 Testy funkcjonalne i akceptacyjne (UI/UX)

Testy manualne przeprowadzono w środowisku docelowym (Windows 10, aplikacja skompilowana `.exe`), aby upewnić się, że zintegrowany stos technologiczny działa poprawnie.

Przeprowadzono walidację następujących scenariuszy użycia:

1. **Pełny cykl wizualizacji:** Wgranie pliku → Wybór algorytmu → Generowanie → Wyświetlenie wyniku. Potwierdzono poprawne działanie wszystkich zaimplementowanych typów wizualizacji (SAT i MaxSAT).

2. **Obsługa dużych plików:** Przetestowano zachowanie aplikacji przy wczytywaniu plików przekraczających 50MB. Potwierdzono, że interfejs (Electron) pozostaje responsywny dzięki asynchronicznemu przetwarzaniu w tle (Celery).

3. **Wznawianie pracy (Resume):** Zweryfikowano, czy po zatrzymaniu i wznowieniu wizualizacji (np. *Matrix*), pasek postępu kontynuuje od zapamiętanego momentu, a dane nie są dublowane.

9.3 Testy wydajnościowe i weryfikacja wywłaszczania

Przeprowadzono testy obciążeniowe polegające na wielokrotnym uruchamianiu i natychmiastowym przerywaniu (Stop/Start) zadań obliczeniowych. Potwierdzono, że worker typu `solo` na systemie Windows poprawnie zwalnia zasoby pamięci i nie ulega zawieszeniu ("zombie process") po przerwaniu zadania przez użytkownika.

9.3.1 Klasyfikacja wizualizacji pod kątem wywłaszczania

Ze względu na różnorodną złożoność obliczeniową algorytmów (od liniowej $O(n)$ do wykładniczej $O(2^n)$), zachowanie mechanizmu pauzowania różni się w zależności od wybranego typu wizualizacji. Wyróżniono trzy kategorie.

Kategoria 1: Wizualizacje standardowe (Długotrwałe, przerywalne)

Do tej grupy należą wizualizacje o średniej i wysokiej złożoności obliczeniowej, przetwarzające duże zbiory danych w przewidywalnych pętlach:

- *Heatmap*,
- *Distribution Chart*,
- *Resolution Graph*,
- *Matrix*,
- *Tree*,
- *Cluster*.

Charakterystyka: W przypadku tych algorytmów czas generowania dla dużych plików wejściowych jest zauważalny dla użytkownika (od kilkunastu sekund do kilku minut). Zaimplementowany mechanizm wywłaszczania działa tu w sposób optymalny – użytkownik może w dowolnym momencie wstrzymać proces, a system robi to wtedy w najbliższej możliwej dla tego chwili. Jest to wzorcowy przykład działania systemu.

Kategoria 2: Wizualizacje nisko-opóźnieniowe (Instant)

Grupa ta obejmuje algorytmy o złożoności liniowej lub silnie zoptymalizowanej, które przetwarzają strukturę pliku CNF "w locie":

- *Interaction Graph*,
- *2-Clauses Interaction Graph*,
- *Direct Graphical Model*,
- *Factor Graph*.

Charakterystyka: Dzięki optymalizacji parsera, wizualizacje te generują się zazwyczaj w czasie poniżej 1 sekundy. Mimo że mechanizm wywłaszczania (Time-Based Throttling) jest w nich zaimplementowany, w praktyce użytkownik rzadko ma okazję z niego skorzystać, gdyż zadanie kończy się, zanim nastąpi interakcja. Mechanizm ten pełni tu rolę "bezpiecznika"na wypadek wczytania ekstremalnie dużych plików wejściowych.

Kategoria 3: Wyjątek algorytmiczny – DPLL Solver

Wizualizacja działania solvera DPLL stanowi wyjątek, w którym celowo zrezygnowano z możliwości wywłaszczania w trakcie trwania kluczowych faz obliczeń.

Przyczyna techniczna: Algorytm DPLL wykorzystuje heurystykę MOMS (*Maximum Occurrence in clauses of Minimum Size*), która wymaga sortowania list klauzul. Natywna metoda `list.sort()` w Pythonie jest operacją atomową (wykonywaną w C), której nie można przerwać. Próba zastąpienia jej wolniejszym, "przerywalnym"odpowiednikiem w Pythonie skutkowała drastycznym spadkiem wydajności (czas obliczeń wzrósł o ok. 400%).

Rozwiązanie UX: Uznano, że wydajność jest priorytetem. Zamiast mechanizmu pauzowania, zastosowano prewencyjny komunikat w interfejsie użytkownika. Przed uruchomieniem wizualizacji DPLL system wyświetla ostrzeżenie:

> *"Uwaga: Procesu tego nie można zatrzymać manualnie po uruchomieniu. Program musi samodzielnie dokończyć obliczenia."*

Dzięki temu użytkownik jest świadomy specyfiki tego konkretnego zadania.

9.3.2 Podsumowanie

Zaprojektowany system zarządzania zadaniami stanowi kompromis (ang. *trade-off*) pomiędzy ograniczeniami platformy Windows a wymaganiami użytkowymi. Dzięki zastosowaniu hybrydowego podejścia – pełnego wywłaszczania dla zadań liniowych oraz ostrzeżeń UX dla zadań atomowych – udało się uzyskać system stabilny i przewidywalny dla użytkownika końcowego.

Rozdział 10

Implementacja nowych formuł wizualizacji

10.1 CDCL Solver – wizualizacja działania solvera SAT z uczeniem na konfliktach

10.1.1 Cel i zakres metody

Głównym celem niniejszego modułu jest dekonstrukcja i wizualizacja procesów zachodzących wewnątrz nowoczesnych solverów problemu spełnialności formuł logicznych (SAT). Algorytm CDCL (Conflict-Driven Clause Learning) stanowi fundament współczesnych systemów dowodzenia twierdzeń, a jego zrozumienie wymaga analizy dynamicznych interakcji między przeszukiwaniem przestrzeni stanów a wnioskowaniem na podstawie napotkanych sprzeczności.

10.1.2 Implementacja Algorytmu CDCL

W celu wizualizacji procesów zachodzących wewnątrz algorytmu zdecydowaliśmy się na zaimplementowanie podstawowej jego wersji od zera. Nasz algorytm generuje ślad w postaci GraphEdge i GraphNode, które są później transoformowane w plik typu json i przesyłane do wizualizacji. Sam algorytm można podzielić na:

- Decision: Wybór zmiennej zgodnie z wybraną heurystyką (o czym w sekcji poniższej).
- Unit Propagation: Propagacja wymuszonych przypisań (z wykorzystaniem grafu implikacji).
- Conflict Analysis: Analiza grafu po napotkaniu sprzeczności, znalezienie UIP (Unique Implication Point) i generowanie klauzuli wyuczonej.
- Backjumping: Powrót do poziomu decyzyjnego, na którym wyuczona klauzula staje się jednostkowa.

10.1.3 Heurystyki Wyboru Zmiennych

1. VSIDS (Variable State Independent Decaying Sum) Heurystyka skupiająca się na historii konfliktów.

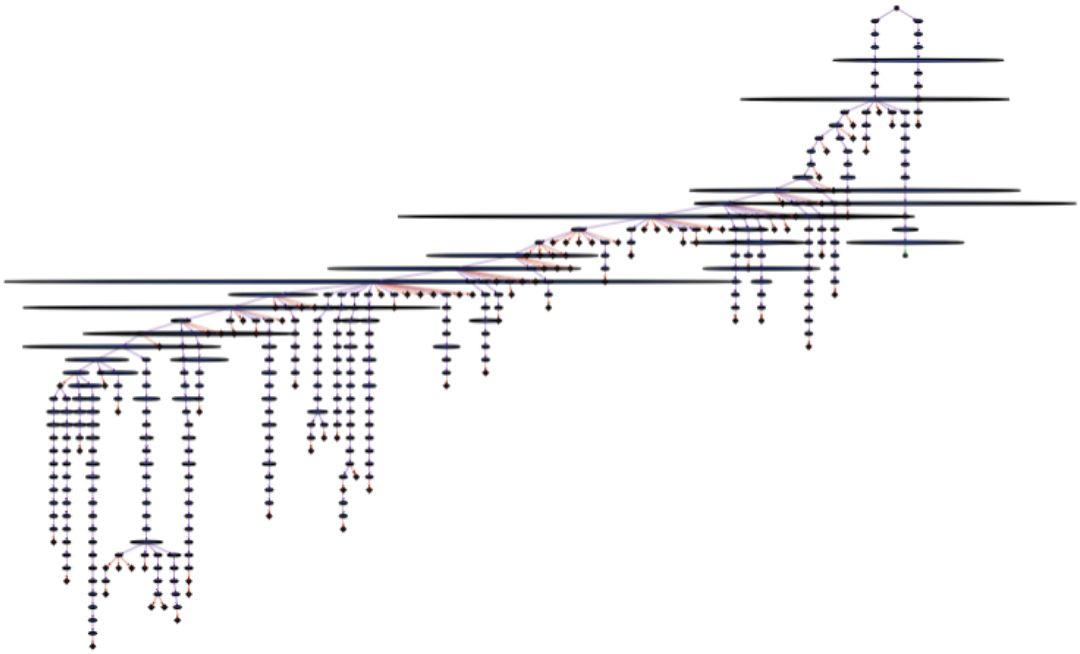
- Każda zmienna posiada licznik aktywności.
- Wystąpienie zmiennej w analizie konfliktu zwiększa jej wagę.
- Zastosowano mechanizm decay (wygaszanie), który promuje zmienne biorące udział w niedawnych konfliktach.
- Phase Saving: Zapamiętywanie ostatniej wartości zmiennej przed cofnięciem (backtrack), co stabilizuje proces poszukiwania.

2. DLIS (Dynamic Largest Individual Sum) Heurystyka zachłanna (greedy).
   - Wybiera zmienną, która występuje w największej liczbie klauzul aktualnie niespełnionych.
   - Wymaga przeliczania statystyk w każdym kroku decyzyjnym, co obrazuje kosztowność obliczeniową podejścia "brute-force"w porównaniu do VSIDS.

10.1.4 Wizualizacja

Kluczowym elementem jest dynamiczny graf drzewa decyzji.

- Wierzchołki (Nodes): Reprezentują stany (Start, Decyzja, Propagacja, Konflikt, SAT).
- Krawędzie (Edges): Symbolizują przejścia logiczne. Zastosowano kolory i etykiety dla odróżnienia decyzji od implikacji.
- Układ Hierarchiczny: Wykorzystano hierarchical layout biblioteki vis-network do automatycznego pozycjonowania węzłów na poziomach decyzyjnych (*level*).



Rysunek 10.1: Wizualizacja CDCL Solver

10.2 Kontrfaktyczna wizualizacja decyzji (What-if)

10.2.1 Cel i zakres metody

Celem zaimplementowanej metody jest interaktywna analiza semantycznych konsekwencji wybranych decyzji logicznych w formule CNF. Moduł pozwala użytkownikowi na symulację przypisania zmiennej do wartości 0 (False) oraz 1 (True) i natychmiastowe porównanie efektów tych decyzji bez trwałej modyfikacji stanu solvera. Metoda ta wspiera proces zrozumienia propagacji jednostkowej oraz identyfikacji "silnych"decyzji prowadzących do szybkich konfliktów lub dużych redukcji przestrzeni poszukiwań.

10.2.2 Architektura rozwiązania

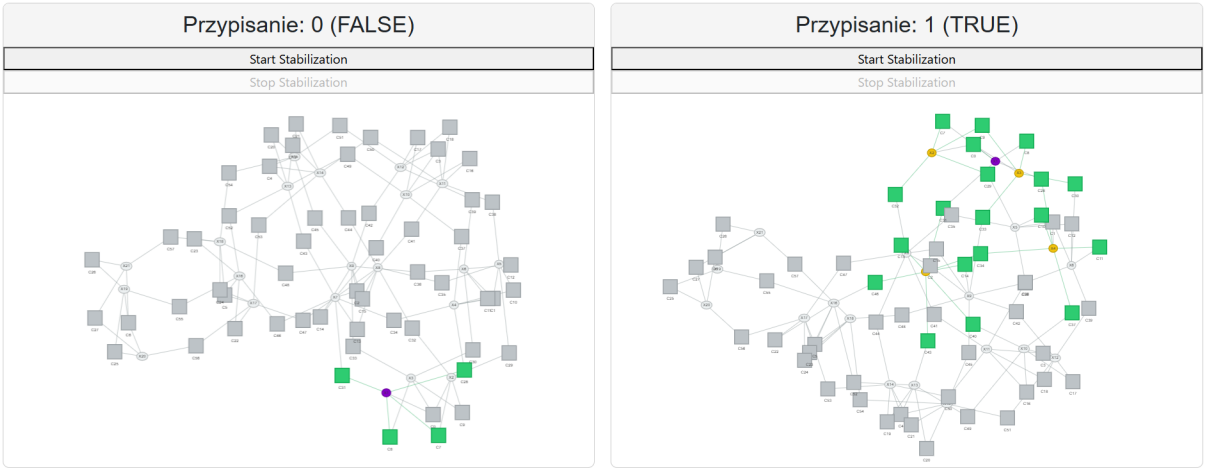
- Backend (Python 3.6 PySAT):
  - Wykorzystano bibliotekę PySAT do niskopoziomowej operacji na formule logicznej.
  - Zastosowano metodę propagate() na obiekcie solvera (np. Glucose), co pozwala na przeprowadzenie Unit Propagation dla konkretnego literału w sposób nieniszczący (bez zmiany bazowej formuły).
  - Dane wynikowe są agregowane do struktury grafu dwudzielnego (Factor Graph), gdzie węzły reprezentują zmienne i klauzule.
- Frontend (Angular Network-vis)
  - Zastosowano widok Side-by-Side, wyświetlający jednocześnie dwa niezależne grafy dla obu wariantów decyzji.
  - Wykorzystano dynamiczne aktualizowanie zbiorów danych DataSet z biblioteki vis.js, co pozwala na płynne przełączanie między analizowanymi zmiennymi bez przeładowywania całej aplikacji.

10.2.3 Algorytm działania

1. Inicjalizacja: Wczytanie pliku CNF z bazy danych, inicjalizacja solvera i wygenerowanie listy dostępnych zmiennych.
2. Probing: Dla wybranej zmiennej x, solver wykonuje dwa przejścia propagacji:
   - $propagate(x)$ – skutki przyjęcia prawdy.
   - $propagate(\neg x)$ – skutki przyjęcia fałszu.
3. Mapowanie wizualne:
   - Węzły zmiennych: Kolorowane w zależności od stanu (zmienna decyzyjna - czerwona, zmienna wymuszona przez propagację - niebieski i żółty, zmienna neutralna - szary).
   - Węzły klauzul: Wyróżnienie klauzul spełnionych - kolor zielony oraz klauzul jeszcze nie spełnionych - szary.

10.2.4 Kluczowe funkcjonalności interfejsu

- Selektor zmiennych: Pozwala na wybór dowolnej zmiennej z formuły do analizy.
- Synchronizacja wizualna: Zastosowanie identycznych układów (layout) dla obu grafów w celu ułatwienia dostrzeżenia różnic w strukturze powiązań.
- Wskaźniki stabilizacji: Monitoring procesu pozycjonowania węzłów (Physics Stabilization) dla zapewnienia czytelności grafu.



Rysunek 10.2: Kontrfaktyczna wizualizacja zmiennych (what-if)

10.3 Trudność Semantyczna Zmiennych

10.3.1 Cel i koncepcja

Wprowadzono mechanizm identyfikacji zmiennych krytycznych dla procesu rozstrzygania formuły SAT, wykraczający poza analizę czysto strukturalną. Metoda pozwala na odróżnienie zmiennych, które są jedynie gęsto połączone w grafie, od tych, które faktycznie generują najwięcej konfliktów i wymuszają najgłębsze powroty (backjumping) solvera CDCL (z heurystyką VSIDS).

10.3.2 Instrumentacja Solvera (Backend)

Rozszerzono zaimplementowany solver CDCL o moduł Semantic Profiler, który w czasie rzeczywistym gromadzi statystyki dla każdej zmiennej podczas przebiegu algorytmu:

- Conflict Participation: Liczba wystąpień zmiennej w nowo wygenerowanych klauzulach uczących (związek z heurystyką VSIDS).
- Backjump Impact: Sumaryczna głębokość skoków wstecznych wywołanych przez błędne decyzje na danej zmiennej.
- Propagation Yield: Średnia liczba implikacji (Unit Propagation) generowanych przez przypisanie danej zmiennej.

10.3.3 Przetwarzanie i Agregacja danych

Dane surowe są normalizowane przy użyciu skali logarytmicznej (w celu niwelacji ekstremalnych pików aktywności) do końcowego wskaźnika trudności $S \in [0, 1]$.

- Wzór agregacji: Wynik jest sumą ważoną trzech powyższych składowych:

$$S(x_i) = \alpha \ln(1 + C_i) + \beta \ln(1 + B_i) + \gamma \left(\frac{P_i}{\max(1, D_i)} \right) \tag{10.1}$$

  Gdzie:

  $S(x_i)$ — wyjściowy wynik trudności semantycznej (*Semantic Score*) dla zmiennej x_i,

  C_i — całkowita liczba wystąpień zmiennej w klauzulach konfliktowych (udział w konfliktach),

  B_i — suma dystansów skoków wstecznych (*backjump distance*) przypisanych do decyzji na zmiennej x_i,

  P_i — łączna liczba literaleów przypisanych w procesie *Unit Propagation* po decyzji na x_i,

  D_i — liczba operacji wyboru (*decisions*) zmiennej x_i przez heurystykę rozgałęziającą,

  α, β, γ — wagi komponentów (np. $0.5, 0.3, 0.2$), określające wpływ poszczególnych cech na końcowy ranking.

$$Score_{final}(x_i) = \frac{S(x_i)}{\max_{j \in V} S(x_j)} \tag{10.2}$$

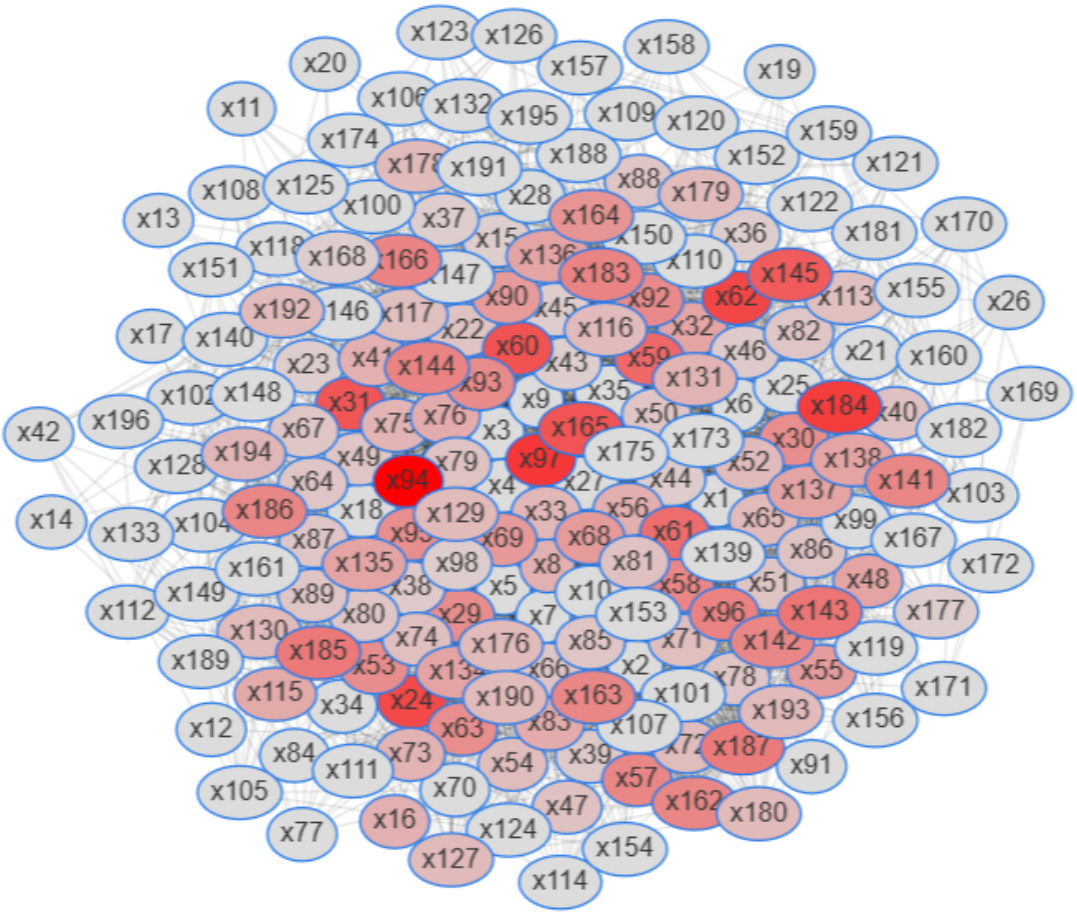
- Mapowanie kolorów: Wartość S jest mapowana na paletę barw (Heatmap) w formacie RGB, gdzie kolor żółty/jasnoszary oznacza zmienne o niskim wpływie, a intensywna czerwień zmienne krytyczne.

10.3.4 Wizualizacja (Frontend)

Wyniki są prezentowane w aplikacji Angularowej za pomocą biblioteki vis-network (Interaction Graph). Graf jest tworzony w tym samym momencie, co wykonywany jest algorytm CDCL i można go oglądać w widoku przeznaczonym do wizualizacji solvera CDCL.

- Węzły (Nodes): Reprezentują zmienne. Kolor tła węzła jest dynamicznie nadawany przez backend na podstawie obliczonej trudności semantycznej.

- Krawędzie (Edges): Łączą zmienne występujące w tych samych klauzulach, tworząc kontekst strukturalny dla danych semantycznych.



Rysunek 10.3: Wizualizacja trudności semantycznej zmiennych

10.4 Wizualizacja Przestrzeni Stanów i Krajobrazu Kosztów (SAT Landscape)

10.4.1 Cel Metody

Metoda ma na celu przekształcenie abstrakcyjnej, wysokowymiarowej przestrzeni przypisań zmiennych logicznych w intuicyjną mapę 2D. Pozwala ona na wizualną ocenę struktury semantycznej formuły SAT, identyfikację "dolin"satysfakcji (regionów bliskich spełnienia) oraz "szczytów"konfliktów. Dodatkowym celem jest analiza zachowania solvera poprzez nałożenie jego trajektorii na wygenerowany krajobraz.

10.4.2 Architektura Rozwiązania

Proces generowania wizualizacji opiera się na trzech filarach:

1. Próbkowanie Przestrzeni (Sampling):
    - Dla małych formuł ($n \leq 11$) stosowany jest algorytm brute-force (pełny iloczyn kartezjański), co gwarantuje kompletny obraz przestrzeni.

- Dla dużych instancji stosowane jest próbkowanie losowe w celu uzyskania statystycznej reprezentacji tła.

2. Funkcja Kosztu (Fitness Function):

   - Zdefiniowano znormalizowaną funkcję kosztu $f(A) = \frac{U}{C}$, gdzie U to liczba niespełnionych klauzul dla przypisania A, a C to całkowita liczba klauzul. Wartość 0 (kolor zielony) reprezentuje pełną satysfakcję (SAT).

3. Redukcja Wymiarowości (Dimensionality Reduction):

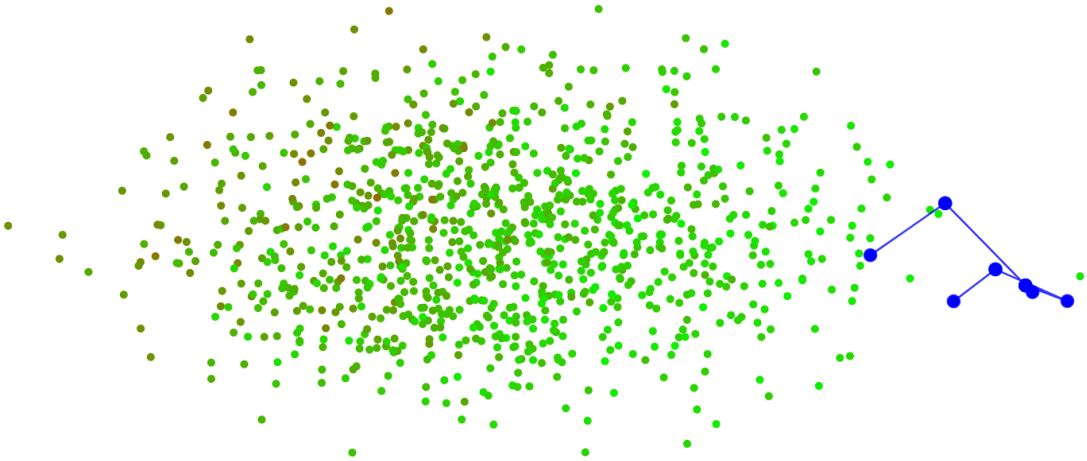
   - Wysokowymiarowe wektory binarne (przypisania) są rzutowane do przestrzeni 2D przy użyciu algorytmu PCA (Principal Component Analysis). Pozwala to na zachowanie globalnej struktury danych i relacji między przypisaniami w czytelnej formie graficznej.

10.4.3 Integracja z Solverem (Trajektoria)

Unikalną cechą metody jest możliwość śledzenia procesu decyzyjnego solvera PySAT (Glucose 3). Poprzez mechanizm propagacji założeń (unit propagation), system rejestruje stany pośrednie (modele cząstkowe), przez które przechodzi solver w drodze do znalezienia rozwiązania. Stany te są rzutowane na ten sam układ współrzędnych co tło, tworząc liniową trajektorię.

10.4.4 Implementacja Techniczna

- Backend: Asynchroniczne zadania realizowane przez Celery, co zapobiega blokowaniu wątku głównego aplikacji przy przetwarzaniu dużych formuł.
- Frontend: Interaktywny wykres typu hybrid scatter-line chart zrealizowany w Chart.js.
  - Warstwa Tła: Dynamicznie kolorowane punkty (gradient od zielonego do czerwonego) reprezentujące gęstość ograniczeń.
  - Warstwa Ścieżki: Niebieska linia łącząca kolejne kroki solvera, pozwalająca ocenić, czy algorytm efektywnie omija regiony wysokiego kosztu.



Rysunek 10.4: Wizualizacja krajobrazu spełnialności formuły (SAT Landscape)

Rozdział 11

Podsumowanie i wnioski końcowe

Celem niniejszej pracy było zaprojektowanie i implementacja systemu do wizualizacji formuł logicznych (SAT i MaxSAT), który łączyłby intuicyjność interfejsu graficznego z wydajnością przetwarzania algorytmicznego. Realizacja projektu wymagała rozwiązania szeregu problemów natury architektonicznej oraz systemowej, wynikających ze specyfiki środowiska Windows oraz ograniczeń języka Python w zastosowaniach wielowątkowych.

Głównym osiągnięciem projektu jest stworzenie stabilnego środowiska uruchomieniowego, które integruje technologie webowe (Django, Angular) w formę samodzielnej aplikacji desktopowej (Electron). Kluczowe zrealizowane funkcjonalności to:

- **Wszechstronność wizualizacji:** Zaimplementowano obsługę szerokiego spektrum algorytmów wizualizacyjnych, od prostych reprezentacji macierzowych, przez grafy interakcji, aż po złożone drzewa decyzyjne solvera DPLL. Rozszerzono system o obsługę problemu MaxSAT, uwzględniając wagi klauzul w wizualizacjach *Factor Graph* i *Resolution Graph*.

- **Zarządzanie długotrwałymi procesami:** Opracowano i wdrożono hybrydowy mechanizm wywłaszczania zadań oparty o strategię *Time-Based Throttling*. Pozwoliło to na zachowanie responsywności aplikacji nawet podczas przetwarzania plików CNF o dużej złożoności, przy jednoczesnym minimalnym narzucie obliczeniowym.

- **Łatwość dystrybucji:** Dzięki wykorzystaniu narzędzia PyInstaller, skomplikowany stos technologiczny (Python, Redis, Celery) został ukryty przed użytkownikiem końcowym wewnątrz jednego pliku wykonywalnego, niewymagającego konfiguracji.

- **Nowe formuły wizualizacji**: Oprócz tego zaimplementowano 4 nowe formuły wizualizacji formuł logicznych:

  - **Wizualizacja CDCL Solver:** Pozwala na wgląd w mikro-decyzje algorytmu, wizualizując dynamiczne powstawanie klauzul uczących i mechanizm *backjumpingu*. Dzięki temu użytkownik może zaobserwować, jak heurystyki takie jak VSIDS czy DLIS wpływają na efektywność przeszukiwania drzewa decyzji.
  - **Kontrfaktyczna wizualizacja decyzji (What-if):** Umożliwia bezpieczną eksplorację konsekwencji decyzji przed ich podjęciem. Widok Side-by-Side oraz integracja z metodą *propagate()* solvera PySAT dostarcza natychmiastowej informacji o sile propagacji jednostkowej dla poszczególnych literałów.
  - **Wizualizacja trudności semantycznej zmiennych:** Wprowadza wskaźnik *Semantic Score*, który nadaje danym strukturalnym nowy wymiar. Mapowanie

kolorystyczne pozwala na błyskawiczną identyfikację zmiennych krytycznych, które generują najwięcej konfliktów i mają największy wpływ na stabilność formuły.

- **Krajobraz spełnialności formuły (SAT Landscape):** Przenosi proces na poziom makro, mapując wysokowymiarową przestrzeń stanów do formy topograficznej 2D za pomocą algorytmu PCA. Dodatkowo nakłada trajektorię solvera na krajobraz spełnialności.